10-casestudies

Casestudy-1:

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

1. Problem Statement

Design a University Scholarship Allocation System that evaluates applicants based on aggregate marks, subject-wise minimum marks, attendance percentage, and annual family income. The system must identify eligible students, rank them according to predefined rules, resolve ties fairly, and select scholarship recipients according to the available number of seats.

2. Objectives

1. Evaluate applicants using academic performance, attendance, and income criteria.
2. Verify the minimum required marks in every subject.
3. Reject applicants who fail to meet any eligibility requirement.
4. Rank eligible applicants by aggregate marks.
5. Resolve ties using attendance and family income.
6. Select the required number of scholarship recipients.
7. Display the final ranked list of selected students.

3. Assumptions

1. Each applicant has marks for the same set of subjects.
2. Subject marks and minimum marks are provided on a scale of 0–100.
3. Every subject must meet its specified minimum score.
4. Attendance is expressed as a percentage.
5. Annual family income is provided as a non-negative amount.
6. The minimum attendance and maximum family income are predefined.
7. Scholarship seats are a non-negative integer.
8. Ranking priority is:
    - Higher aggregate marks first.
    - Higher attendance next.
    - Lower annual family income next.
    - Student ID in ascending order as the final tie-breaker.

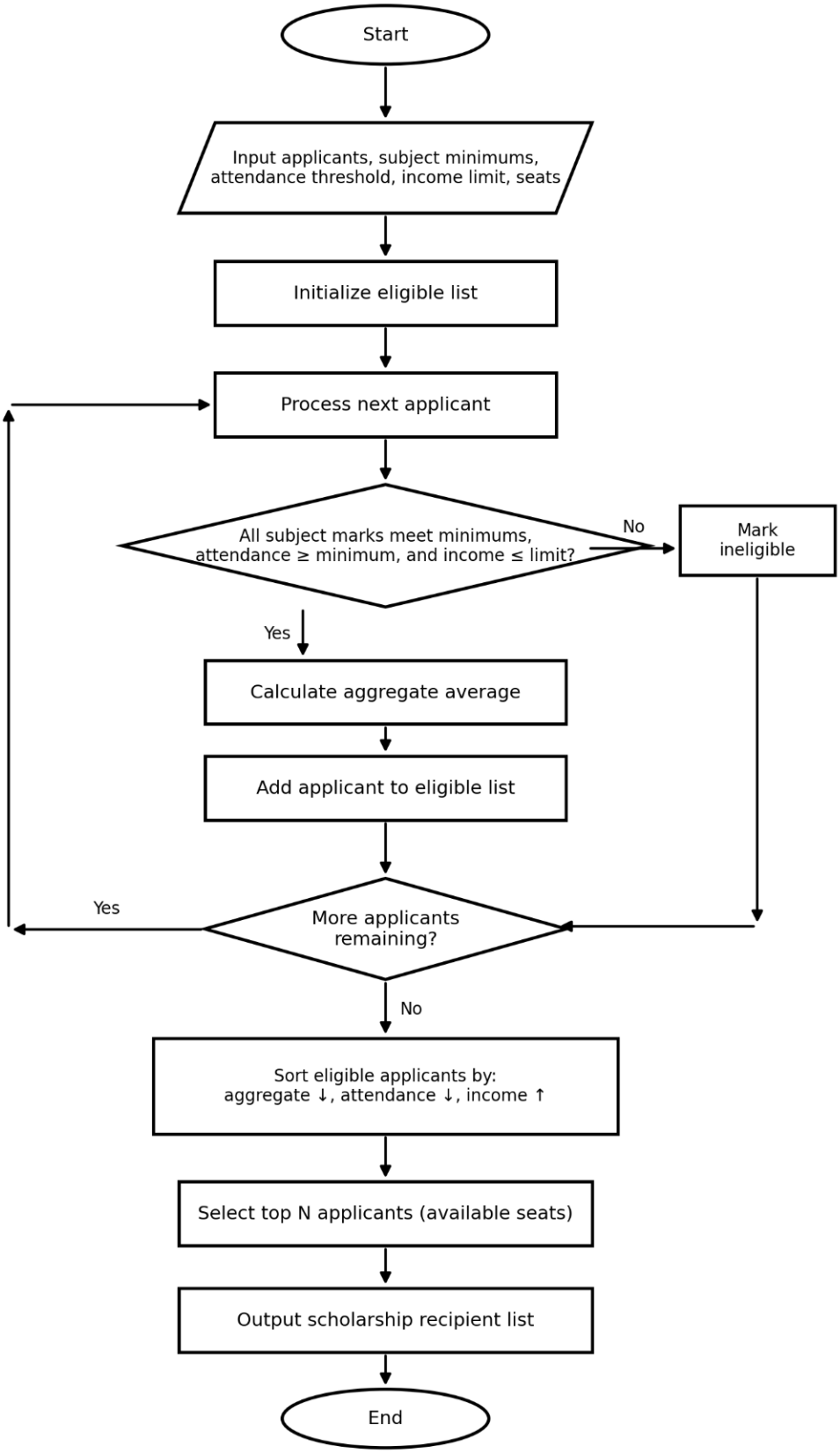
9. Aggregate marks are calculated as the average of the subject marks.

4. Algorithm

1. Start.
2. Read applicant details, subject-wise minimum marks, minimum attendance, maximum income, and available scholarship seats.
3. Initialize an empty list of eligible applicants.
4. For each applicant:
    - Check whether every subject mark meets the required minimum.
    - Check whether attendance meets the minimum requirement.
    - Check whether family income is within the permitted limit.
    - If any condition fails, mark the applicant as ineligible.
    - Otherwise, calculate the average aggregate marks and add the applicant to the eligible list.
5. Sort eligible applicants using these priorities:
    - Aggregate marks in descending order.
    - Attendance in descending order.
    - Annual family income in ascending order.
    - Student ID in ascending order.
6. Select the top applicants up to the available number of seats.
7. Display the selected scholarship recipients.
8. Stop

5. Flowchart

UNIVERSITY SCHOLARSHIP ALLOCATION FLOWCHART



6.Python source code:

```
minimum_marks = 60
minimum_attendance = 75
maximum_income = 200000
name = input("Enter student name: ")
mark1 = int(input("Enter marks in Subject 1: "))
mark2 = int(input("Enter marks in Subject 2: "))
mark3 = int(input("Enter marks in Subject 3: "))
attendance = float(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))
total = mark1 + mark2 + mark3
aggregate = total / 3
if (mark1 >= minimum_marks and
    mark2 >= minimum_marks and
    mark3 >= minimum_marks and
    attendance >= minimum_attendance and
    income <= maximum_income):
    print("Student is eligible for scholarship")
    print("Student Name:", name)
    print("Aggregate Marks:", aggregate)
else:
    print("Student is not eligible for scholarship")
```

7. Pseudocode

```
BEGIN

SET minimum_marks = 60
SET minimum_attendance = 75
SET maximum_income = 200000

INPUT student name
INPUT marks of three subjects
INPUT attendance
INPUT annual family income

total = mark1 + mark2 + mark3
aggregate = total / 3

IF mark1 >= minimum_marks AND
   mark2 >= minimum_marks AND
   mark3 >= minimum_marks AND
```

```
   attendance >= minimum_attendance AND
   income <= maximum_income THEN

    DISPLAY "Student is eligible"
    DISPLAY aggregate

ELSE

    DISPLAY "Student is not eligible"

END IF

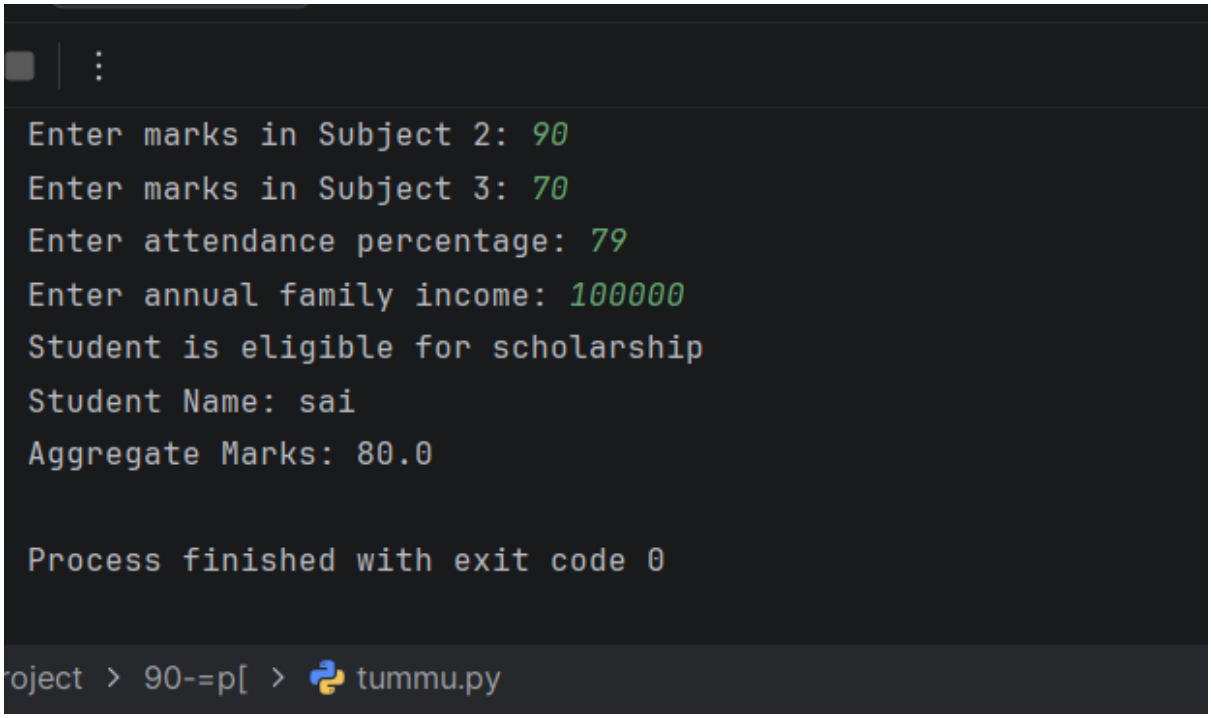
END
```

8.Testcases:

Enter student name: Ravi
Enter marks in Subject 1: 80
Enter marks in Subject 2: 90
Enter marks in Subject 3: 70
Enter attendance percentage: 85
Enter annual family income: 150000
Student is eligible for scholarship
Student Name: Ravi
Aggregate Marks: 80.0

Enter student name: Rahul
Enter marks in Subject 1: 55
Enter marks in Subject 2: 90
Enter marks in Subject 3: 80
Enter attendance percentage: 90
Enter annual family income: 100000
Student is not eligible for scholarship

9.Output screenshot:



10.Conclusion:

The University Scholarship Allocation System helps evaluate students fairly according to academic performance, attendance, and family income. It reduces manual effort and supports transparent scholarship decisions.

Casestudy2:

Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

1. Problem Statement

To develop a Python program that calculates household electricity bills based on electricity consumption slabs, fixed charges, surcharges, and rebates. The system also compares electricity bills across several months and identifies unusual increases in the bill amount.

2. Objective

- Calculate electricity charges using different consumption slabs.
- Add fixed charges and surcharges.
- Apply rebates when applicable.
- Compare monthly electricity bills.

- Identify unusual increases in electricity bill**s.**

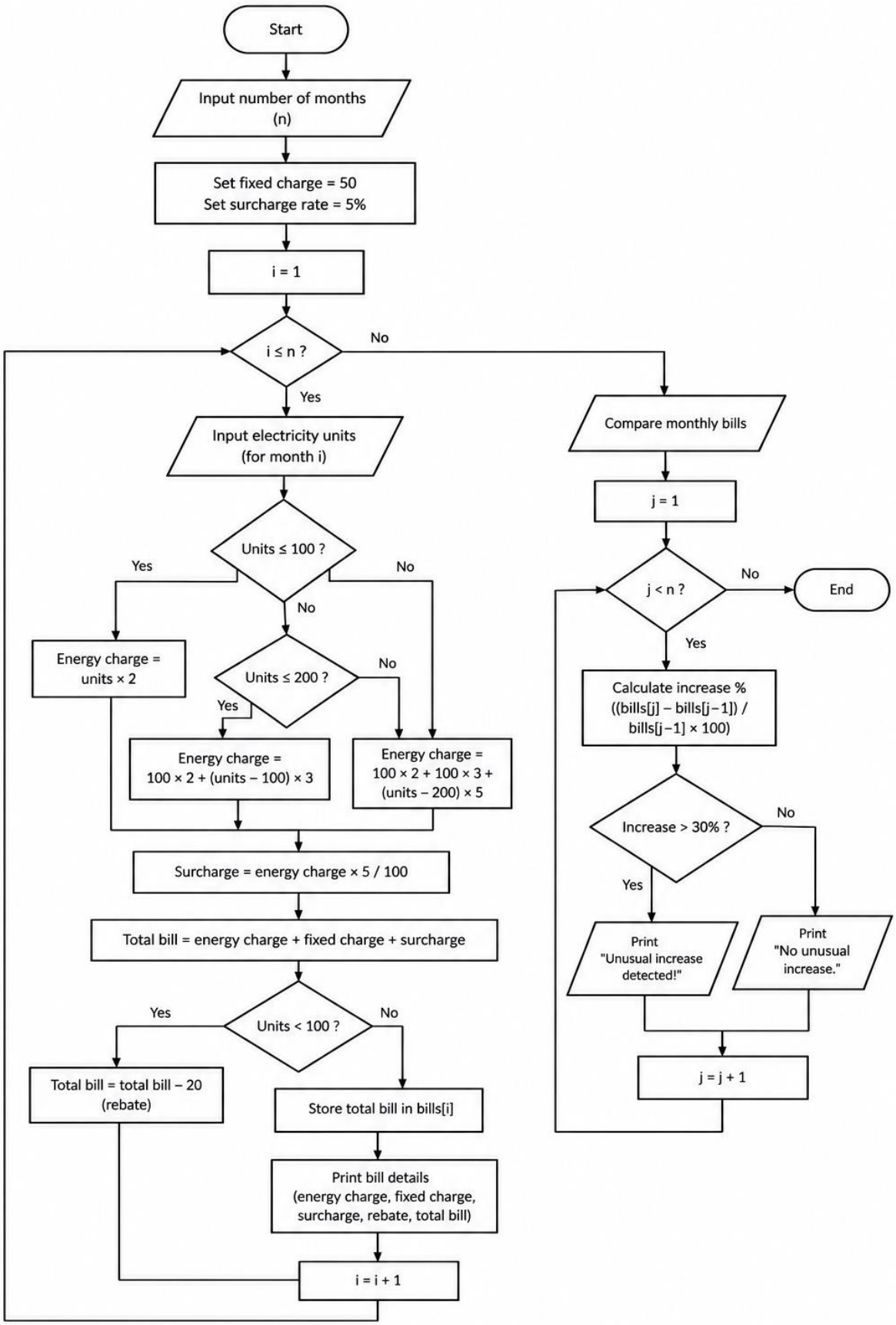
3. Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units (101–200): ₹3 per unit.
- Above 200 units: ₹5 per unit.
- Fixed charge: ₹50 per month.
- Surcharge: 5% of the energy charge.
- Rebate: ₹20 if consumption is below 100 units.
- An increase greater than 30% compared with the previous month's bill is considered unusual.

4. Algorithm

1. Start the program.
2. Enter electricity consumption in units.
3. Calculate energy charges using the consumption slabs.
4. Add the fixed charge.
5. Calculate and add the surcharge.
6. Apply the rebate if consumption is below 100 units.
7. Display the total electricity bill.
8. Repeat for multiple months.
9. Compare each month's bill with the previous month's bill.
10. Display a warning if the bill increases by more than 30%.
11. Stop.

5. Flowchart :


Start
Input number of months (n)
Set fixed charge = 50
Set surcharge rate = 5%
i = 1
i ≤ n ?
No
Yes
Input electricity units (for month i)
Units ≤ 100 ?
Yes
No
Energy charge = units × 2
Units ≤ 200 ?
No
Yes
Energy charge = 100 × 2 + (units − 100) × 3
Energy charge = 100 × 2 + 100 × 3 + (units − 200) × 5
Surcharge = energy charge × 5 / 100
Total bill = energy charge + fixed charge + surcharge
Units < 100 ?
Yes
No
Total bill = total bill − 20 (rebate)
Store total bill in bills[i]
Print bill details (energy charge, fixed charge, surcharge, rebate, total bill)
i = i + 1
Compare monthly bills
j = 1
j < n ?
No
End
Yes
Calculate increase % ((bills[j] − bills[j−1]) / bills[j−1] × 100)
Increase > 30% ?
No
Yes
Print "Unusual increase detected!"
Print "No unusual increase."
j = j + 1

6. Pseudocode

```
START

SET fixed_charge = 50
SET surcharge_rate = 5%

INPUT electricity units

IF units <= 100
   energy_charge = units * 2
ELSE IF units <= 200
   energy_charge = 100 * 2 + (units - 100) * 3
ELSE
   energy_charge = 100 * 2 + 100 * 3
                 + (units - 200) * 5

surcharge = energy_charge * 5 / 100

total_bill = energy_charge + fixed_charge + surcharge

IF units < 100
   total_bill = total_bill - 20

DISPLAY total_bill

COMPARE bills for different months

IF bill increase > 30%
   DISPLAY "Unusual increase detected"

STOP
```

7.Python Source Code

```
fixed_charge = 50
bills = []
for month in range(1, 4):
  print("\nMonth", month)
  units = int(input("Enter electricity units: "))
   if units <= 100:
       energy_charge = units * 2
    elif units <= 200:
       energy_charge = (100 * 2) + (units - 100) * 3
    else:
       energy_charge = (100 * 2) + (100 * 3) + (units - 200) * 5
   surcharge = energy_charge * 5 / 100
```

```
    total_bill = energy_charge + fixed_charge + surcharge
    if units < 100:
        total_bill = total_bill - 20
    bills.append(total_bill)
    print("Energy Charge:", energy_charge)
    print("Fixed Charge:", fixed_charge)
    print("Surcharge:", surcharge)
    print("Total Electricity Bill:", round(total_bill, 2))
    print("\nMonthly Bill Comparison")
    for i in range(1, 3):
    previous_bill = bills[i - 1]
    current_bill = bills[i]
    print("Month", i + 1, "Bill:", round(current_bill, 2))
     if previous_bill > 0:
      increase = ((current_bill - previous_bill)
                / previous_bill) * 100
      if increase > 30:
            print("Warning: Unusual increase detected!")
        else:
            print("No unusual increase detected.")
```

8. Test Cases

#1
Month 1
Enter electricity units: 80
Energy Charge: 160
Fixed Charge: 50
Surcharge: 8.0
Total Electricity Bill: 198.0
#2
Month 1
Enter electricity units: 150
Energy Charge: 350
Fixed Charge: 50
Surcharge: 17.5
Total Electricity Bill: 417.5

9.output screenshot:



10.conclusion:

The Progressive Electricity Billing System calculates electricity bills using consumption slabs, fixed charges, surcharges, and rebates. It also compares monthly bills and warns users about unusual increases, helping households monitor their electricity expenses.

Case study 3:

Employee Payroll and Deduction Analysis: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

1. Problem Statement

To develop a Python program that calculates an employee's gross salary using basic pay, allowances, overtime, and incentives. The system applies progressive deductions, calculates the net salary, and generates a formatted salary statement.

2. Objective

- Calculate gross salary from basic pay, allowances, overtime, and incentives.
- Apply progressive deductions based on gross salary.
- Calculate the final net salary.
- Display a formatted salary statement.
- Help employees understand their earnings and deductions.

3. Assumptions

For this example, the salary and deduction rules are:

- Basic pay: Entered by the user.
- Allowances: 20% of basic pay.
- Overtime pay: Number of overtime hours × ₹200.
- Incentives: Entered by the user.
- Progressive deductions:

Gross salary range	Deduction rate
First ₹10,000	5%
Next ₹10,000 (₹10,001–₹20,000)	10%
Amount above ₹20,000	15%

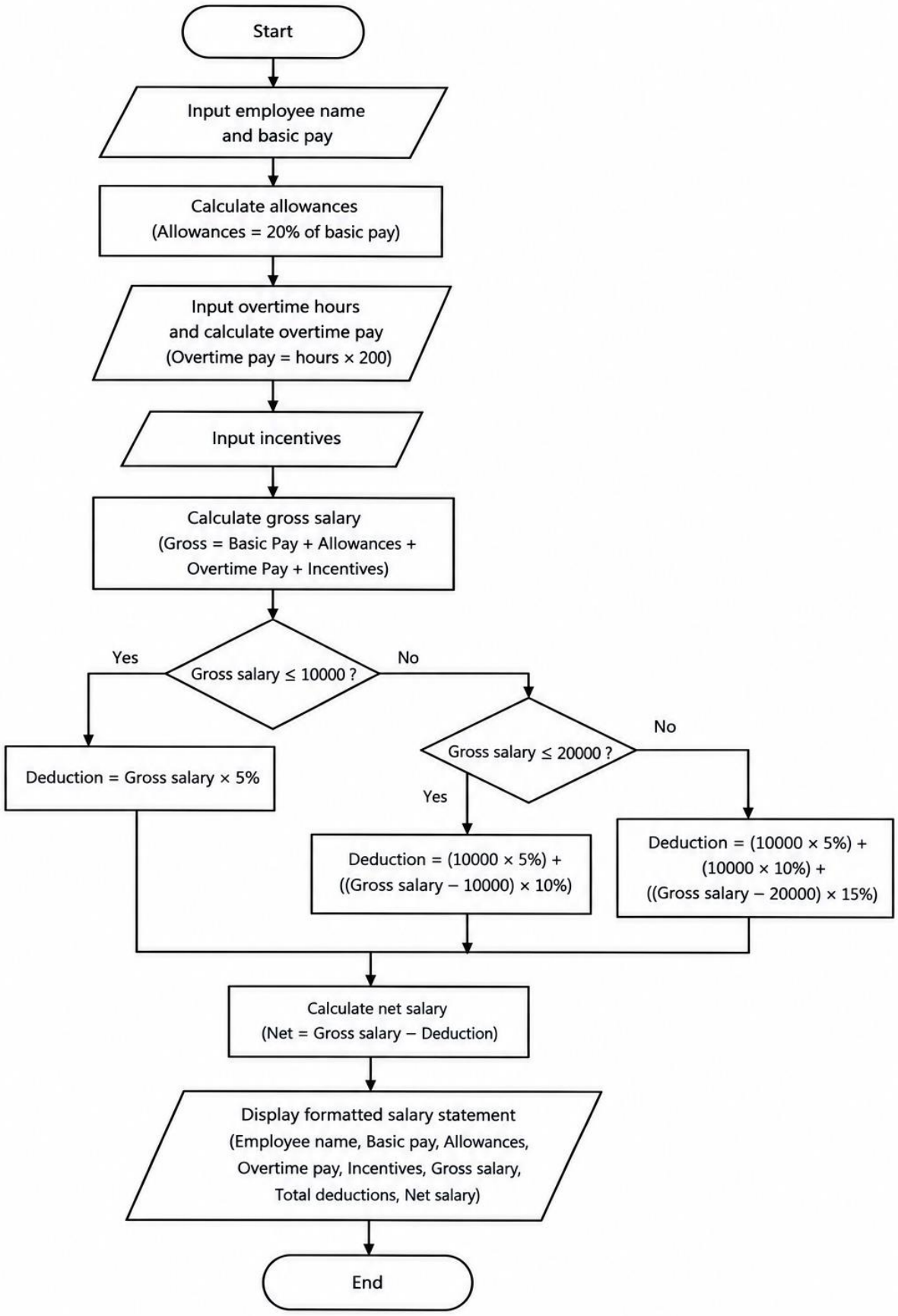
- Net salary = Gross salary − Total deductions.

These are example rules for an academic project, not actual statutory payroll or tax rates.

4. Algorithm

1. Start the program.
2. Enter the employee's name and basic pay.
3. Calculate allowances as 20% of basic pay.
4. Enter overtime hours and calculate overtime pay.
5. Enter the incentive amount.
6. Calculate gross salary by adding basic pay, allowances, overtime pay, and incentives.
7. Calculate progressive deductions using the given salary slabs.
8. Calculate net salary by subtracting deductions from gross salary.
9. Display the formatted salary statement.
10. Stop.

5. Flowchart Image:


Start
Input employee name and basic pay
Calculate allowances (Allowances = 20% of basic pay)
Input overtime hours and calculate overtime pay (Overtime pay = hours × 200)
Input incentives
Calculate gross salary (Gross = Basic Pay + Allowances + Overtime Pay + Incentives)
Gross salary ≤ 10000 ?
Yes
No
Deduction = Gross salary × 5%
Gross salary ≤ 20000 ?
Yes
No
Deduction = (10000 × 5%) + ((Gross salary − 10000) × 10%)
Deduction = (10000 × 5%) + (10000 × 10%) + ((Gross salary − 20000) × 15%)
Calculate net salary (Net = Gross salary − Deduction)
Display formatted salary statement (Employee name, Basic pay, Allowances, Overtime pay, Incentives, Gross salary, Total deductions, Net salary)
End

6. Pseudocode:

```
START

INPUT employee name
INPUT basic pay

allowances = basic pay * 20 / 100

INPUT overtime hours
overtime pay = overtime hours * 200

INPUT incentives

gross salary = basic pay + allowances
             + overtime pay + incentives

IF gross salary <= 10000
   deduction = gross salary * 5 / 100
ELSE IF gross salary <= 20000
   deduction = 10000 * 5 / 100
             + (gross salary - 10000) * 10 / 100
ELSE
   deduction = 10000 * 5 / 100
             + 10000 * 10 / 100
             + (gross salary - 20000) * 15 / 100

net salary = gross salary - deduction

DISPLAY salary statement

STOP
```

7.Python Source Code:

```
name = input("Enter employee name: ")
basic_pay = float(input("Enter basic pay: "))
allowances = basic_pay * 20 / 100
overtime_hours = float(input("Enter overtime hours: "))
overtime_pay = overtime_hours * 200
incentives = float(input("Enter incentives: "))
gross_salary = basic_pay + allowances + overtime_pay + incentives
if gross_salary <= 10000:
   deduction = gross_salary * 5 / 100

elif gross_salary <= 20000:
   deduction = (10000 * 5 / 100) + (
      (gross_salary - 10000) * 10 / 100
```

```
    )

else:
    deduction = (10000 * 5 / 100) + (
        10000 * 10 / 100
    ) + ((gross_salary - 20000) * 15 / 100)

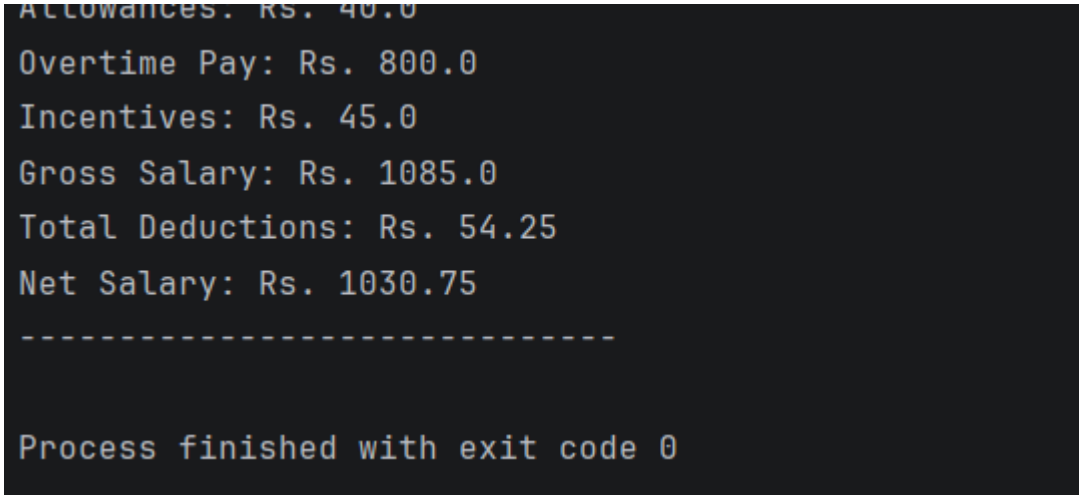

net_salary = gross_salary - deduction
print("Employee Name:", name)
print("Basic Pay: Rs.", round(basic_pay, 2))
print("Allowances: Rs.", round(allowances, 2))
print("Overtime Pay: Rs.", round(overtime_pay, 2))
print("Incentives: Rs.", round(incentives, 2))
print("Gross Salary: Rs.", round(gross_salary, 2))
print("Total Deductions: Rs.", round(deduction, 2))
print("Net Salary: Rs.", round(net_salary, 2))
```

8. Test Cases

#1
Enter employee name: Rahul
Enter basic pay: 8000
Enter overtime hours: 5
Enter incentives: 500
Employee Name: Rahul
Basic Pay: Rs. 8000.0
Allowances: Rs. 1600.0
Overtime Pay: Rs. 1000.0
Incentives: Rs. 500.0
Gross Salary: Rs. 11100.0
Total Deductions: Rs. 610.0
Net Salary: Rs. 10490.0

#2
Enter employee name: Ravi
Enter basic pay: 15000
Enter overtime hours: 10
Enter incentives: 1000
Employee Name: Ravi
Basic Pay: Rs. 15000.0
Allowances: Rs. 3000.0
Overtime Pay: Rs. 2000.0
Incentives: Rs. 1000.0
Gross Salary: Rs. 21000.0
Total Deductions: Rs. 1650.0
Net Salary: Rs. 19350.0

9.Output screenshot:



10.conclusion:

The Employee Payroll and Deduction Analysis System calculates gross salary, applies progressive deductions, and determines net salary. It generates a formatted salary statement that clearly displays the employee's earnings and deductions.

Case study 4:

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

1. Problem Statement

To develop a Python program that calculates railway fares for multiple passengers based on distance slabs, travel classes, age-based concessions, and reservation fees. The system also calculates cancellation penalties and the final refund amount when a reservation is cancelled.

2. Objectives

- Calculate fares based on distance and travel class.
- Apply concessions based on passenger age.
- Add reservation fees.
- Calculate the total fare for multiple passengers.
- Apply cancellation penalties.
- Display the total fare and refund amount.

3. Assumptions

These are example rules for an academic project, not official railway fares or cancellation policies.

Fare rates per kilometre

Distance slab	Sleeper	AC	First Class
First 100 km	₹1.00	₹2.00	₹3.00
Next 200 km	₹0.80	₹1.50	₹2.50
Above 300 km	₹0.60	₹1.20	₹2.00

Other rules:

- Reservation fee: ₹50 per passenger.
- Children below 5 years: 50% concession on base fare.
- Senior citizens aged 60 or above: 40% concession on base fare.
- Other passengers: No concession.
- Cancellation before departure: 10% penalty on total fare.
- Cancellation after departure: 25% penalty on total fare.

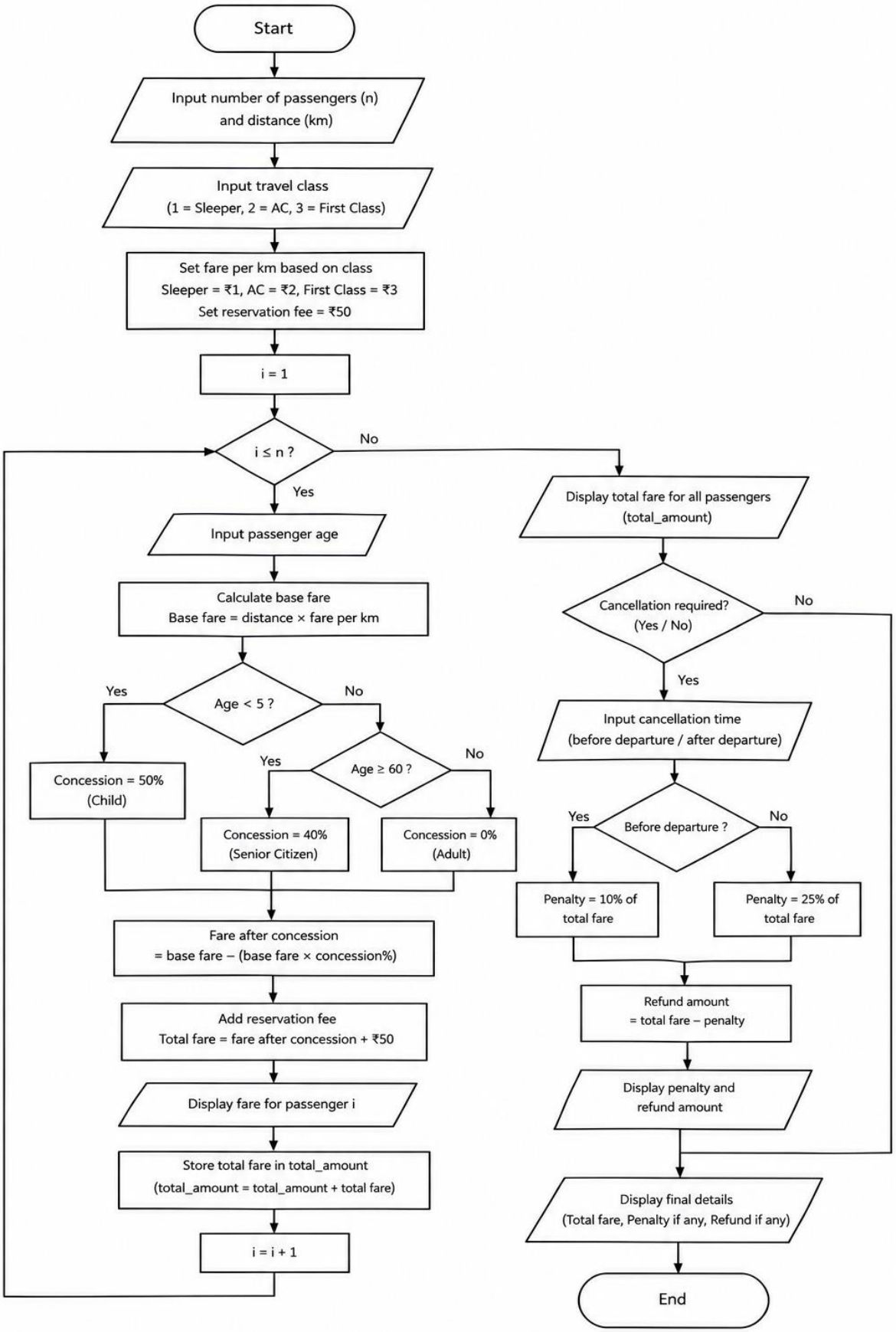
For simplicity, the program treats each passenger's age-based concession independently and applies the cancellation penalty to the combined booking fare.

4. Algorithm

1. Start the program.
2. Enter the number of passengers and journey distance.
3. Select the travel class.
4. Calculate the fare using the distance slabs.
5. For each passenger, enter their name and age.
6. Apply the age-based concession.
7. Add the reservation fee.
8. Display each passenger's fare.
9. Calculate the total fare for all passengers.
10. Ask whether the booking should be cancelled.
11. If cancelled, calculate the penalty based on cancellation timing.
12. Display the penalty, refund amount, and final details.

13. Stop.

5. Flowchart:


Start
Input number of passengers (n) and distance (km)
Input travel class (1 = Sleeper, 2 = AC, 3 = First Class)
Set fare per km based on class Sleeper = ₹1, AC = ₹2, First Class = ₹3 Set reservation fee = ₹50
i = 1
i ≤ n ?
No
Yes
Input passenger age
Calculate base fare Base fare = distance × fare per km
Age < 5 ?
Yes
No
Concession = 50% (Child)
Age ≥ 60 ?
Yes
No
Concession = 40% (Senior Citizen)
Concession = 0% (Adult)
Fare after concession = base fare – (base fare × concession%)
Add reservation fee Total fare = fare after concession + ₹50
Display fare for passenger i
Store total fare in total_amount (total_amount = total_amount + total fare)
i = i + 1
Display total fare for all passengers (total_amount)
Cancellation required? (Yes / No)
No
Yes
Input cancellation time (before departure / after departure)
Before departure ?
Yes
No
Penalty = 10% of total fare
Penalty = 25% of total fare
Refund amount = total fare – penalty
Display penalty and refund amount
Display final details (Total fare, Penalty if any, Refund if any)
End

6. Pseudocode:

```
START

INPUT number of passengers
INPUT journey distance
INPUT travel class

SET total_fare = 0

FOR each passenger

    INPUT passenger name
    INPUT passenger age

    CALCULATE base fare using distance slabs

    IF age < 5
        concession = 50%
    ELSE IF age >= 60
        concession = 40%
    ELSE
        concession = 0%

    fare_after_concession =
        base fare - (base fare * concession / 100)

    passenger_fare = fare_after_concession + 50

    total_fare = total_fare + passenger_fare

    DISPLAY passenger fare

END FOR

DISPLAY total_fare

INPUT cancellation choice

IF booking is cancelled
    INPUT cancellation timing

    IF before departure
        penalty = total_fare * 10 / 100
    ELSE
        penalty = total_fare * 25 / 100

    refund = total_fare - penalty
```

```
    DISPLAY penalty
    DISPLAY refund

ELSE
    DISPLAY booking confirmed

STOP
```

7.Python Source Code :

```
distance = int(input("Enter journey distance in km: "))
number_of_passengers = int(input("Enter number of passengers: "))

travel_class = input(
    "Enter travel class (Sleeper, AC, First Class): "
).lower()
if travel_class == "sleeper":
    rate1 = 1.00
    rate2 = 0.80
    rate3 = 0.60

elif travel_class == "ac":
    rate1 = 2.00
    rate2 = 1.50
    rate3 = 1.20

elif travel_class == "first class":
    rate1 = 3.00
    rate2 = 2.50
    rate3 = 2.00

else:
    print("Invalid travel class")
    exit()
if distance <= 100:
    base_fare = distance * rate1

elif distance <= 300:
    base_fare = (100 * rate1) + ((distance - 100) * rate2)

else:
    base_fare = (
        (100 * rate1)
        + (200 * rate2)
        + ((distance - 300) * rate3)
    )
```

```
reservation_fee = 50
total_fare = 0

for i in range(number_of_passengers):

    print("\nPassenger", i + 1)

    name = input("Enter passenger name: ")
    age = int(input("Enter passenger age: "))
    if age < 5:
        concession = 50

    elif age >= 60:
        concession = 40

    else:
        concession = 0

    discount = base_fare * concession / 100

    passenger_fare = base_fare - discount
    passenger_fare = passenger_fare + reservation_fee

    total_fare = total_fare + passenger_fare

    print("Passenger Name:", name)
    print("Base Fare: Rs.", round(base_fare, 2))
    print("Concession: Rs.", round(discount, 2))
    print("Reservation Fee: Rs.", reservation_fee)
    print("Passenger Fare: Rs.", round(passenger_fare, 2))
   print("\nTotal Booking Fare: Rs.", round(total_fare, 2))
   cancel = input("Do you want to cancel the booking? (yes/no): ").lower()

if cancel == "yes":

    timing = input(
        "Cancellation before departure or after departure? "
    ).lower()

    if timing == "before":
        penalty = total_fare * 10 / 100

    elif timing == "after":
        penalty = total_fare * 25 / 100

    else:
        penalty = 0
```

```
        print("Invalid cancellation timing")

    refund = total_fare - penalty

    print("Cancellation Penalty: Rs.", round(penalty, 2))
    print("Refund Amount: Rs.", round(refund, 2))

else:
    print("Booking confirmed")
    print("Amount Payable: Rs.", round(total_fare, 2))
```

8. Test Cases:

#1
Enter journey distance in km: 150
Enter number of passengers: 2
Enter travel class (Sleeper, AC, First Class): sleeper

Passenger 1
Enter passenger name: Ravi
Enter passenger age: 30
Passenger Name: Ravi
Base Fare: Rs. 140.0
Concession: Rs. 0.0
Reservation Fee: Rs. 50
Passenger Fare: Rs. 190.0

Passenger 2
Enter passenger name: Sita
Enter passenger age: 65
Passenger Name: Sita
Base Fare: Rs. 140.0
Concession: Rs. 56.0
Reservation Fee: Rs. 50
Passenger Fare: Rs. 134.0

Total Booking Fare: Rs. 324.0
Do you want to cancel the booking? (yes/no): no
Booking confirmed
Amount Payable: Rs. 324.0
Enter journey distance in km: 100
Enter number of passengers: 1
Enter travel class (Sleeper, AC, First Class): ac
#2

Enter journey distance in km: 100

Enter number of passengers: 1
Enter travel class (Sleeper, AC, First Class): ac

Passenger 1
Enter passenger name: Rahul
Enter passenger age: 30
Passenger Name: Rahul
Base Fare: Rs. 200.0
Concession: Rs. 0.0
Reservation Fee: Rs. 50
Passenger Fare: Rs. 250.0

Total Booking Fare: Rs. 250.0
Do you want to cancel the booking? (yes/no): yes
Cancellation before departure or after departure? before
Cancellation Penalty: Rs. 25.0
Refund Amount: Rs. 225.0
#3
Enter journey distance in km: 100
Enter number of passengers: 1
Enter travel class (Sleeper, AC, First Class): sleeper

Passenger 1
Enter passenger name: Anil
Enter passenger age: 35
Passenger Name: Anil
Base Fare: Rs. 100.0
Concession: Rs. 0.0
Reservation Fee: Rs. 50
Passenger Fare: Rs. 150.0

Total Booking Fare: Rs. 150.0
Do you want to cancel the booking? (yes/no): yes
Cancellation before departure or after departure? after
Cancellation Penalty: Rs. 37.5
Refund Amount: Rs. 112.5

9.output screenshot:

```
Total Booking Fare: Rs. 2142.56
Do you want to cancel the booking? (yes/no): yes
Cancellation before departure or after departure? after departure
Invalid cancellation timing
Cancellation Penalty: Rs. 0
Refund Amount: Rs. 2142.56

Process finished with exit code 0
```

10.conclusion:

The Railway Fare and Reservation Calculator calculates fares for multiple passengers using distance slabs and travel classes. It applies age-based concessions, adds reservation fees, and calculates cancellation penalties and refunds. This project demonstrates the use of Python conditions, loops, arithmetic operations, and user input.

Case study 5:

Retail Billing with Multiple Discount Rules: Process multiple products, quantities, category specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

1. Problem Statement

To develop a Python program that generates a retail bill for multiple products based on product prices, quantities, category-specific taxes, threshold discounts, and maximum concession limits. The system calculates the total amount, applies discounts according to predefined rules, adds applicable taxes, and generates an itemized invoice.

2. Objectives

- Enter multiple products and their quantities.
- Calculate the cost of each product.
- Apply discounts when the purchase reaches specified thresholds.
- Apply category-specific taxes.
- Ensure discounts do not exceed the maximum concession limit.
- Generate an itemized invoice showing subtotal, discount, tax, and final amount.

3. Assumptions

These are example rules for an academic project, not actual tax advice.

Product categories and tax rates

Category	Tax rate
Grocery	5%
Clothing	12%
Electronics	18%

Discount rules

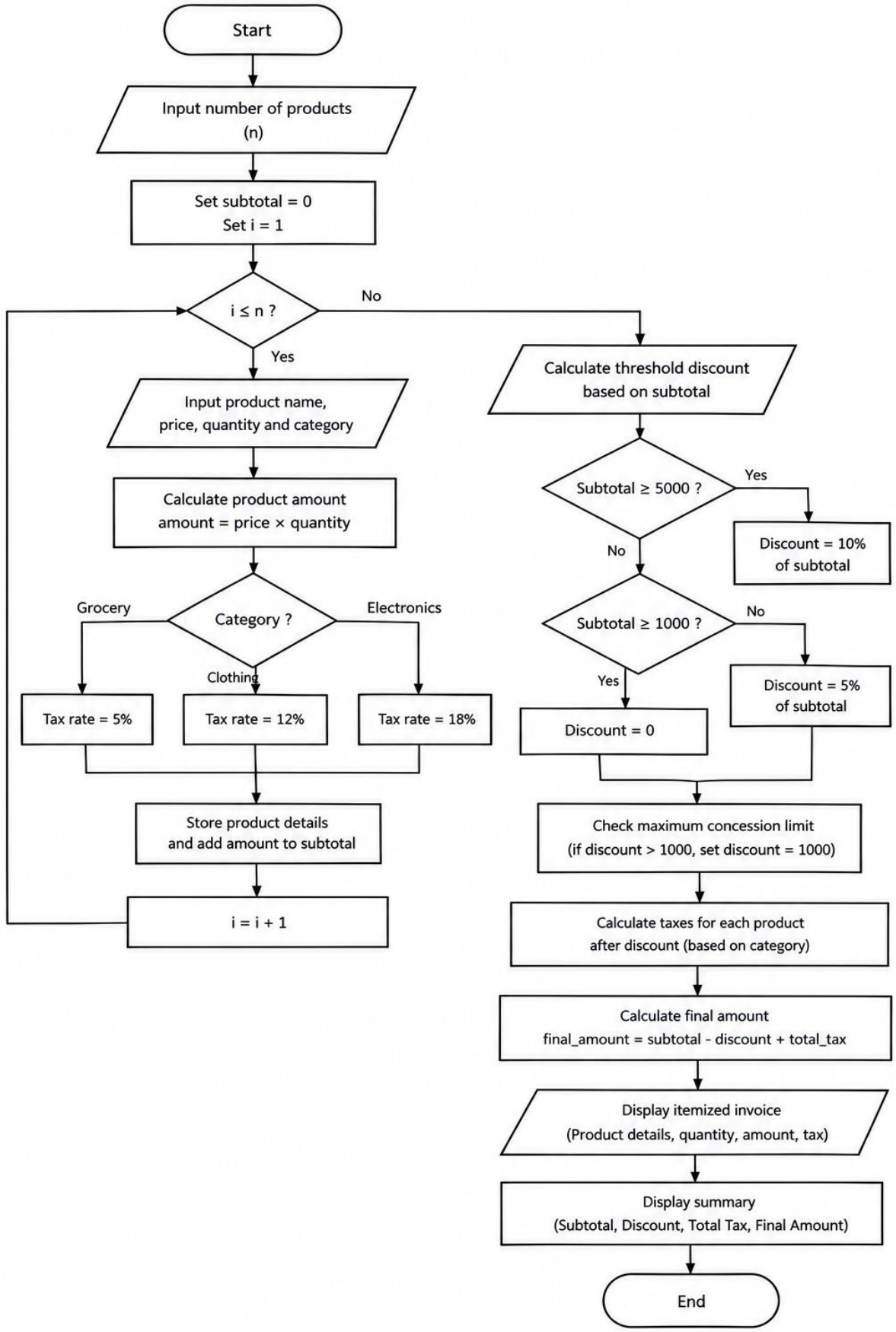
- Purchase subtotal of ₹1,000 or more: 5% discount.
- Purchase subtotal of ₹5,000 or more: 10% discount.
- Maximum discount allowed: ₹1,000.
- Only one threshold discount applies: the highest eligible rate.
- Tax is calculated on the amount after discount.
- Product prices are entered before tax.

Sketching it out36%

4. Algorithm

1. Start the program.
2. Enter the number of products.
3. For each product, enter its name, price, quantity, and category.
4. Calculate the product amount by multiplying price by quantity.
5. Apply the appropriate tax rate based on the product category.
6. Add all product amounts to calculate the subtotal before tax.
7. Check the subtotal and apply the appropriate threshold discount.
8. Limit the discount to a maximum of ₹1,000.
9. Calculate the tax for each product after allocating the discount proportionally.
10. Calculate the final bill by adding taxes and subtracting the discount.
11. Display the itemized invoice with product details, subtotal, discount, taxes, and final amount.
12. Stop

5. Flowchart Image:


Start
Input number of products (n)
Set subtotal = 0
Set i = 1
i ≤ n ?
No
Yes
Input product name, price, quantity and category
Calculate product amount
amount = price × quantity
Category ?
Grocery
Clothing
Electronics
Tax rate = 5%
Tax rate = 12%
Tax rate = 18%
Store product details and add amount to subtotal
i = i + 1
Calculate threshold discount based on subtotal
Subtotal ≥ 5000 ?
Yes
Discount = 10% of subtotal
No
Subtotal ≥ 1000 ?
No
Yes
Discount = 5% of subtotal
Discount = 0
Check maximum concession limit
(if discount > 1000, set discount = 1000)
Calculate taxes for each product after discount (based on category)
Calculate final amount
final_amount = subtotal - discount + total_tax
Display itemized invoice
(Product details, quantity, amount, tax)
Display summary
(Subtotal, Discount, Total Tax, Final Amount)
End

6. Pseudocode:

```
START

INPUT number of products
SET subtotal = 0

FOR each product
   INPUT product name
   INPUT price
   INPUT quantity
   INPUT category

   product_amount = price * quantity
   subtotal = subtotal + product_amount
END FOR

IF subtotal >= 5000
   discount = subtotal * 10 / 100
ELSE IF subtotal >= 1000
   discount = subtotal * 5 / 100
ELSE
   discount = 0

IF discount > 1000
   discount = 1000

CALCULATE tax for each product after discount

final_amount = subtotal - discount + total_tax

DISPLAY itemized invoice
DISPLAY subtotal
DISPLAY discount
DISPLAY total tax
DISPLAY final amount

STOP

final_amount = subtotal - discount + total_tax

DISPLAY itemized invoice
DISPLAY subtotal
DISPLAY discount
DISPLAY total tax
DISPLAY final amount

STOP
```

7.Python Source Code:

```
products = []
number = int(input("Enter number of products: "))
subtotal = 0
for i in range(number):

    print("\nProduct", i + 1)

    name = input("Enter product name: ")
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    category = input(
        "Enter category (Grocery, Clothing, Electronics): "
    ).lower()

    amount = price * quantity
    if category == "grocery":
        tax_rate = 5
    elif category == "clothing":
        tax_rate = 12
    elif category == "electronics":
        tax_rate = 18
    else:
        print("Invalid category. Using 0% tax.")
        tax_rate = 0
    products.append([
        name, price, quantity, amount, tax_rate
    ])

    subtotal = subtotal + amount
if subtotal >= 5000:
    discount = subtotal * 10 / 100
elif subtotal >= 1000:
    discount = subtotal * 5 / 100
else:
    discount = 0
if discount > 1000:
    discount = 1000
total_tax = 0
print("Product       Qty    Amount    Tax")

for product in products:

    name = product[0]
    quantity = product[2]
    amount = product[3]
```

```
    tax_rate = product[4]
    item_discount = discount * amount / subtotal

    discounted_amount = amount - item_discount

    tax = discounted_amount * tax_rate / 100

    total_tax = total_tax + tax

    print(name, "   ", quantity, "   "
          round(amount, 2), "   ", round(tax, 2))
final_amount = subtotal - discount + total_tax
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Total Tax: Rs.", round(total_tax, 2))
print("Final Amount: Rs.", round(final_amount, 2))
```

8. Test Cases:

#1
Enter number of products: 2

Product 1
Enter product name: Rice
Enter product price: 500
Enter quantity: 2
Enter category (Grocery, Clothing, Electronics): grocery

Product 2
Enter product name: Shirt
Enter product price: 1000
Enter quantity: 1
Enter category (Grocery, Clothing, Electronics): clothing
Product Qty Amount Tax
Rice 2 1000.0 47.5
Shirt 1 1000.0 114.0
Subtotal: Rs. 2000.0
Discount: Rs. 100.0
Total Tax: Rs. 161.5
Final Amount: Rs. 2061.5
#2
Enter number of products: 1

Product 1
Enter product name: Laptop
Enter product price: 3000
Enter quantity: 2

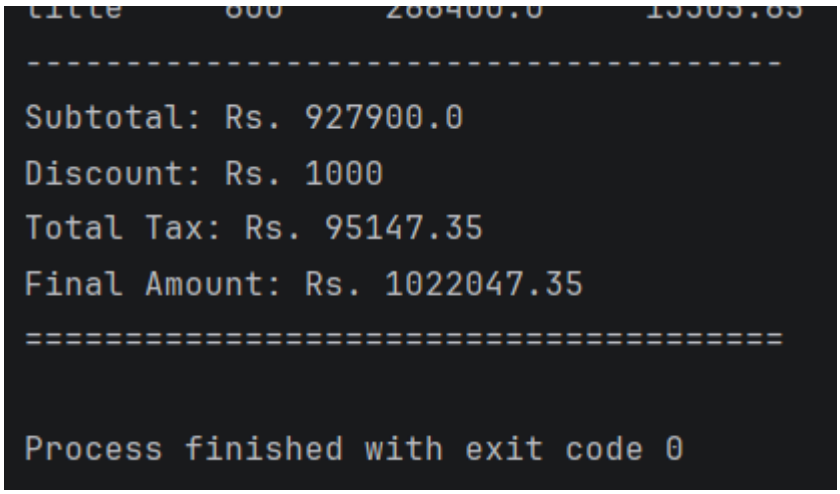
Enter category (Grocery, Clothing, Electronics): electronics

Product Qty Amount Tax
Laptop 2 6000.0 972.0
Subtotal: Rs. 6000.0
Discount: Rs. 600.0
Total Tax: Rs. 972.0
Final Amount: Rs. 6372.0
#3

Enter number of products: 1

Product 1
Enter product name: Television
Enter product price: 15000
Enter quantity: 1
Enter category (Grocery, Clothing, Electronics): electronics
Product Qty Amount Tax
Television 1 15000.0 2520.0
Subtotal: Rs. 15000.0
Discount: Rs. 1000
Total Tax: Rs. 2520.0
Final Amount: Rs. 16520.0

9.output screenshot:



10.conclusion:

The Retail Billing System processes multiple products, calculates category-specific taxes, applies threshold discounts, and enforces a maximum discount limit. It generates an itemized invoice showing the subtotal, discount, tax, and final payable amount.

Case study 6:

Manufacturing Cost and Profitability Analysis: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

1. Problem Statement

To develop a Python program that calculates the total manufacturing cost of a product by considering material costs, labour charges, wastage, and machine costs. The system calculates the selling price, determines the profit, and identifies whether production meets a specified profit target.

2. Objectives

- Calculate the total cost of raw materials.
- Calculate labour and machine charges.
- Include material wastage costs.
- Calculate total manufacturing cost and cost per unit.
- Determine selling revenue and profit.
- Check whether the production meets the specified profit target.

3. Assumptions

These are example values for an academic project.

- Material cost is entered by the user.
- Labour cost is calculated using labour hours × hourly rate.
- Wastage cost is calculated as a percentage of material cost.
- Machine cost is calculated using machine hours × hourly rate.
- Selling price per unit is entered by the user.
- Profit = Total selling revenue − Total manufacturing cost.
- The profit target is entered by the user.

Example wastage rate: 5% of material cost.

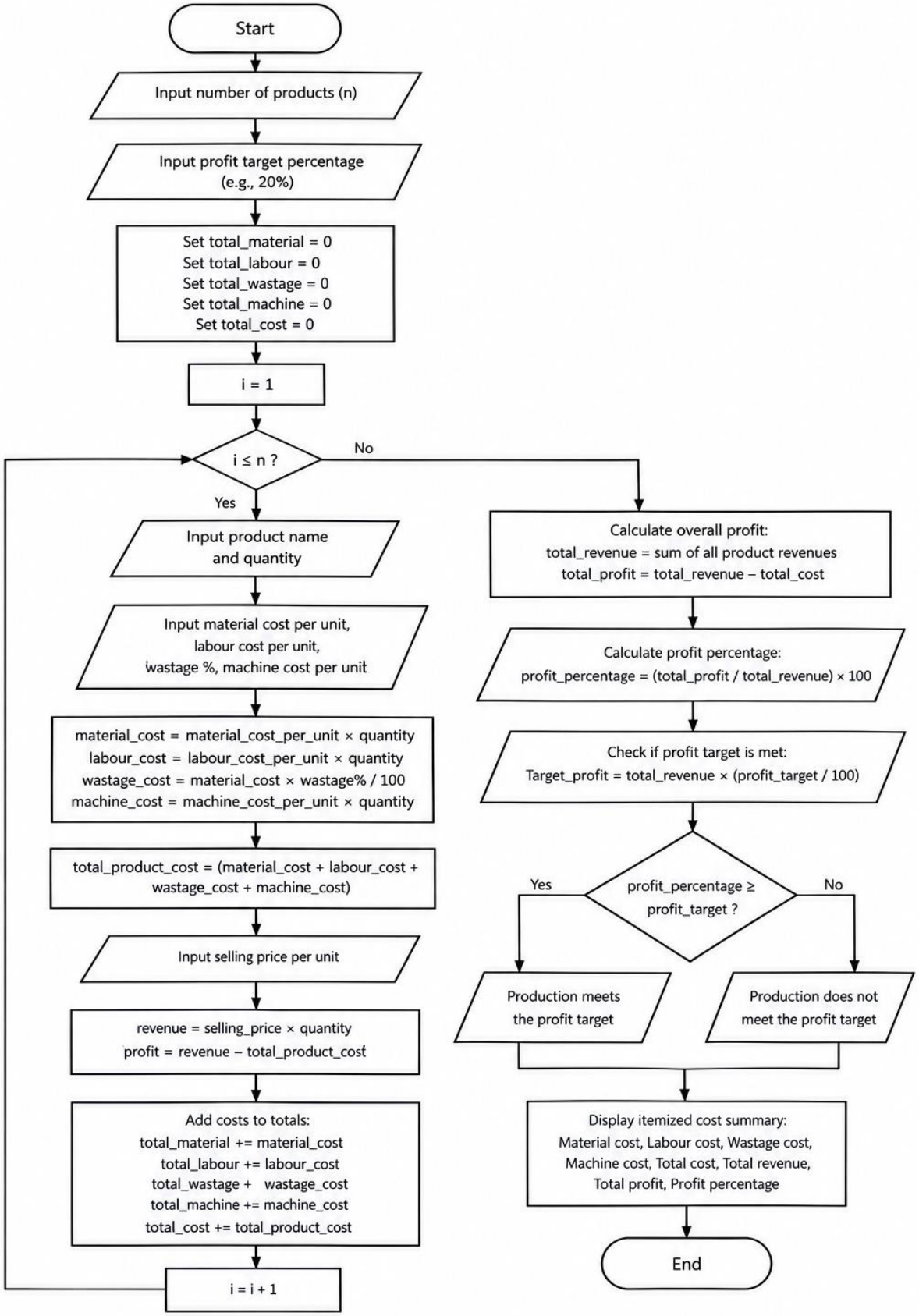
Sketching it out36%

4. Algorithm

1. Start the program.
2. Enter the number of products and the target profit percentage.

3. For each product, enter its name, quantity, material cost per unit, labour cost per unit, machine cost per unit, and selling price per unit.
4. Calculate total material, labour, and machine costs.
5. Calculate wastage cost as 5% of material cost.
6. Add all costs to calculate total manufacturing cost.
7. Calculate total selling revenue.
8. Calculate profit and profit percentage.
9. Compare the profit percentage with the target.
10. Display the cost summary and whether the target is achieved.
11. Stop.

5. Flowchart Image:


Start
Input number of products (n)
Input profit target percentage (e.g., 20%)
Set total_material = 0
Set total_labour = 0
Set total_wastage = 0
Set total_machine = 0
Set total_cost = 0
i = 1
i ≤ n ?
No
Yes
Input product name and quantity
Input material cost per unit, labour cost per unit, wastage %, machine cost per unit
material_cost = material_cost_per_unit × quantity
labour_cost = labour_cost_per_unit × quantity
wastage_cost = material_cost × wastage% / 100
machine_cost = machine_cost_per_unit × quantity
total_product_cost = (material_cost + labour_cost + wastage_cost + machine_cost)
Input selling price per unit
revenue = selling_price × quantity
profit = revenue – total_product_cost
Add costs to totals:
total_material += material_cost
total_labour += labour_cost
total_wastage + wastage_cost
total_machine += machine_cost
total_cost += total_product_cost
i = i + 1
Calculate overall profit:
total_revenue = sum of all product revenues
total_profit = total_revenue – total_cost
Calculate profit percentage:
profit_percentage = (total_profit / total_revenue) × 100
Check if profit target is met:
Target_profit = total_revenue × (profit_target / 100)
profit_percentage ≥ profit_target ?
Yes
No
Production meets the profit target
Production does not meet the profit target
Display itemized cost summary:
Material cost, Labour cost, Wastage cost, Machine cost, Total cost, Total revenue, Total profit, Profit percentage
End

6. Pseudocode:

```
START

INPUT number of products
INPUT target profit percentage

SET total_cost = 0
SET total_revenue = 0

FOR each product

    INPUT product name
    INPUT quantity
    INPUT material cost per unit
    INPUT labour cost per unit
    INPUT machine cost per unit
    INPUT selling price per unit

    material_cost = material cost per unit * quantity
    labour_cost = labour cost per unit * quantity
    machine_cost = machine cost per unit * quantity

    wastage_cost = material_cost * 5 / 100

    product_cost = material_cost + labour_cost
                 + machine_cost + wastage_cost

    revenue = selling price per unit * quantity

    profit = revenue - product_cost

    ADD product_cost to total_cost
    ADD revenue to total_revenue

END FOR

total_profit = total_revenue - total_cost

IF total_cost > 0
    profit_percentage = total_profit / total_cost * 100
ELSE
    profit_percentage = 0

IF profit_percentage >= target profit percentage
    DISPLAY "Profit target achieved"
ELSE
    DISPLAY "Profit target not achieved"
```

DISPLAY total_cost
DISPLAY total_revenue
DISPLAY total_profit
DISPLAY profit_percentage

STOP

7.Python Source Code:

```
number = int(input("Enter number of products: "))
target = float(input("Enter target profit percentage: "))
total_cost = 0
total_revenue = 0
for i in range(number):

    print("\nProduct", i + 1)

    name = input("Enter product name: ")
    quantity = int(input("Enter quantity: "))

    material_per_unit = float(
        input("Enter material cost per unit: ")
    )

    labour_per_unit = float(
        input("Enter labour cost per unit: ")
    )

    machine_per_unit = float(
        input("Enter machine cost per unit: ")
    )

    selling_price = float(
        input("Enter selling price per unit: ")
    )
    material_cost = material_per_unit * quantity
    labour_cost = labour_per_unit * quantity
    machine_cost = machine_per_unit * quantity
    wastage_cost = material_cost * 5 / 100
    product_cost = (
        material_cost + labour_cost
        + machine_cost + wastage_cost
    )
    revenue = selling_price * quantity
    profit = revenue - product_cost
```

```
    total_cost = total_cost + product_cost
    total_revenue = total_revenue + revenue
    print("\nProduct Name:", name)
    print("Material Cost: Rs.", round(material_cost, 2))
    print("Labour Cost: Rs.", round(labour_cost, 2))
    print("Machine Cost: Rs.", round(machine_cost, 2))
    print("Wastage Cost: Rs.", round(wastage_cost, 2))
    print("Manufacturing Cost: Rs.", round(product_cost, 2))
    print("Selling Revenue: Rs.", round(revenue, 2))
    print("Product Profit: Rs.", round(profit, 2))
total_profit = total_revenue - total_cost
if total_cost > 0:
    profit_percentage = total_profit / total_cost * 100
else:
    profit_percentage = 0
print("Total Manufacturing Cost: Rs.", round(total_cost, 2))
print("Total Selling Revenue: Rs.", round(total_revenue, 2))
print("Total Profit: Rs.", round(total_profit, 2))
print("Profit Percentage:", round(profit_percentage, 2), "%")
if profit_percentage >= target:
    print("Production meets the profit target.")
else:
    print("Production does not meet the profit target.")
```

8. Test Cases:

#1

```
Enter number of products: 1
Enter target profit percentage: 20

Product 1
Enter product name: Table
Enter quantity: 10
Enter material cost per unit: 100
Enter labour cost per unit: 20
Enter machine cost per unit: 10
Enter selling price per unit: 200

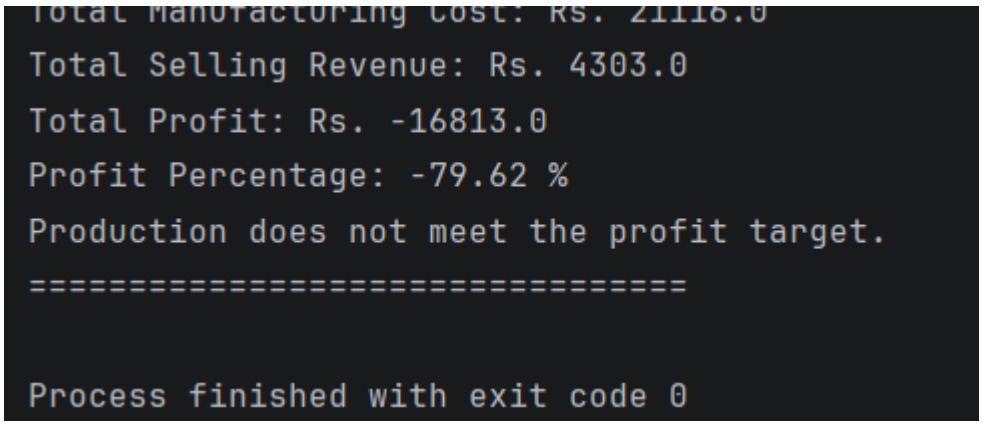
Product Name: Table
Material Cost: Rs. 1000.0
Labour Cost: Rs. 200.0
Machine Cost: Rs. 100.0
Wastage Cost: Rs. 50.0
Manufacturing Cost: Rs. 1350.0
Selling Revenue: Rs. 2000.0
Product Profit: Rs. 650.0
```

Total Manufacturing Cost: Rs. 1350.0
Total Selling Revenue: Rs. 2000.0
Total Profit: Rs. 650.0
Profit Percentage: 48.15 %
Production meets the profit target.
Enter number of products: 1
Enter target profit percentage: 20
#2

Product 1
Enter product name: Chair
Enter quantity: 5
Enter material cost per unit: 100
Enter labour cost per unit: 30
Enter machine cost per unit: 20
Enter selling price per unit: 140

Product Name: Chair
Material Cost: Rs. 500.0
Labour Cost: Rs. 150.0
Machine Cost: Rs. 100.0
Wastage Cost: Rs. 25.0
Manufacturing Cost: Rs. 775.0
Selling Revenue: Rs. 700.0
Product Profit: Rs. -75.0
Total Manufacturing Cost: Rs. 775.0
Total Selling Revenue: Rs. 700.0
Total Profit: Rs. -75.0
Profit Percentage: -9.68 %
Production does not meet the profit target.

9.output screenshot:



10.conclusion:

The Manufacturing Cost and Profitability Analysis System calculates material, labour, wastage, and machine costs. It determines manufacturing cost, selling revenue, and profit, then checks whether production meets the specified profit target. This project demonstrates Python loops, conditional statements, arithmetic operations, and formatted output.

Case study 7:

Smart Parking Fee Management: Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

1. Problem Statement

The Smart Parking Fee Management System is designed to calculate parking charges based on vehicle entry time, exit time, and vehicle category. The system accepts the vehicle type and parking time details from the user and calculates the total parking duration and applicable fee.

It considers different parking rates for different vehicle categories, overnight parking charges, daily maximum limits, and penalty charges for invalid or excessive parking conditions. The system also validates the time inputs to ensure that the entry and exit times are correct.

The main aim is to provide an easy, accurate, and efficient method of calculating parking fees.

2. Objectives

1. To calculate parking charges based on entry and exit times.
2. To support different vehicle categories such as bike, car, and bus.
3. To calculate the total parking duration in hours.
4. To apply additional charges for overnight parking.
5. To implement maximum daily parking fee limits.
6. To apply penalty charges when required.
7. To validate time inputs and prevent invalid entries.
8. To generate the final parking fee receipt.

3. Assumptions

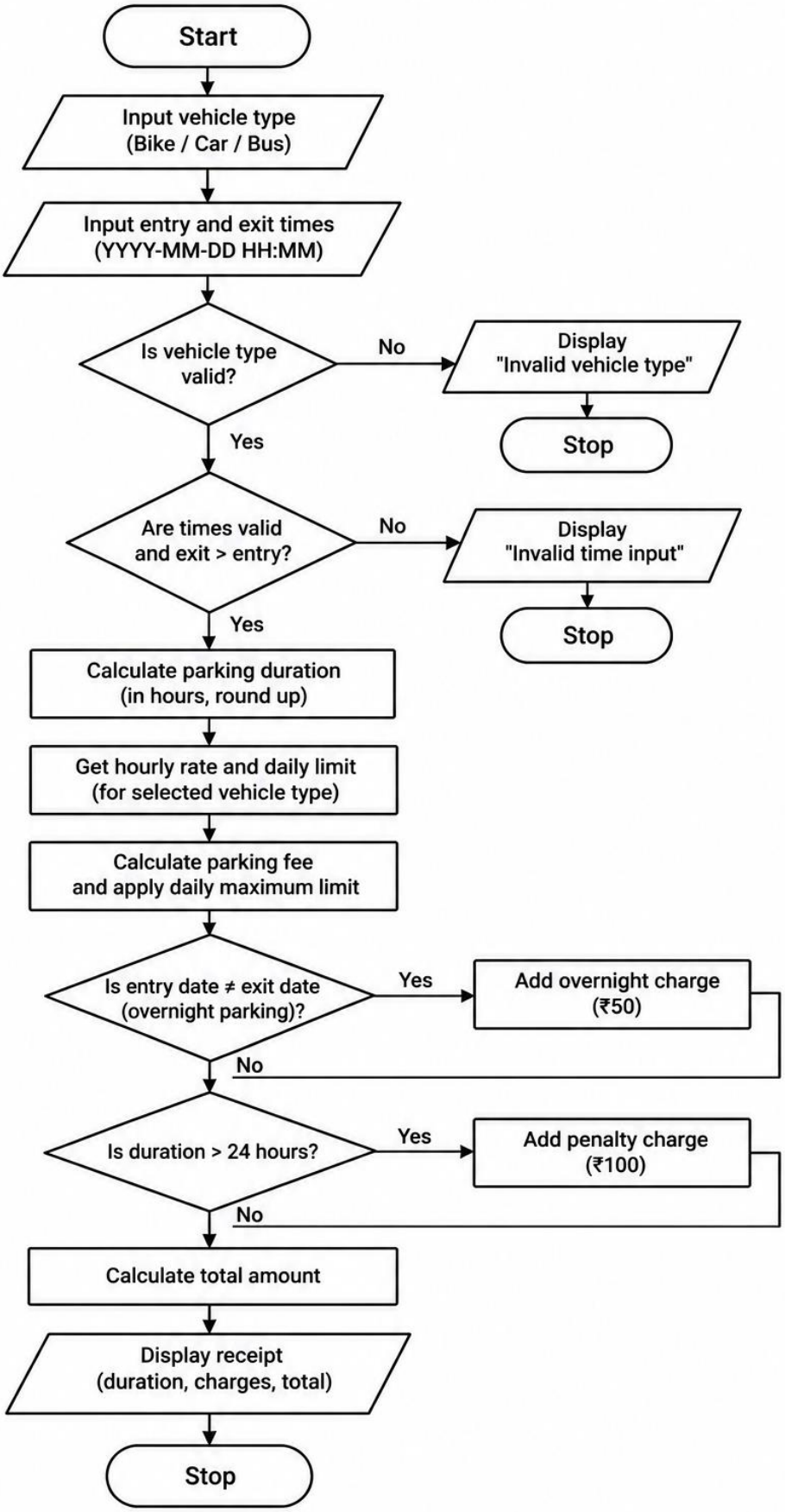
1. The system supports three vehicle categories: Bike, Car, and Bus.
2. Parking charges are calculated per hour.
3. Any extra minutes are rounded up to the next full hour.
4. Entry and exit times are entered in 24-hour format (HH:MM).
5. If exit time is earlier than entry time, it is assumed that the vehicle exited the next day.

6. Overnight parking attracts an additional charge.
7. A maximum daily parking limit is applied.
8. A fixed penalty is charged if the parking duration exceeds 24 hours.
9. The system does not handle multiple days separately; it uses a simple daily-limit calculation.

4. Algorithm

1. Start the program.
2. Display the parking fee management system title.
3. Accept the vehicle category from the user.
4. Check whether the vehicle category is valid.
5. Accept entry time and exit time.
6. Validate the time format.
7. Convert entry and exit times into minutes.
8. Calculate the total parking duration.
9. If exit time is earlier than entry time, consider it as the next day.
10. Convert the parking duration into hours by rounding up extra minutes.
11. Calculate the parking fee according to the vehicle category.
12. Apply the maximum daily fee limit.
13. Check whether overnight parking occurred.
14. Add the overnight charge if applicable.
15. Check whether the parking duration exceeds 24 hours.
16. Add the penalty charge if applicable.
17. Display the total parking duration and final parking fee.
18. Stop the program.

5. Flowchart:


Start
Input vehicle type
(Bike / Car / Bus)
Input entry and exit times
(YYYY-MM-DD HH:MM)
Is vehicle type
valid?
No
Display
"Invalid vehicle type"
Stop
Yes
Are times valid
and exit > entry?
No
Display
"Invalid time input"
Stop
Yes
Calculate parking duration
(in hours, round up)
Get hourly rate and daily limit
(for selected vehicle type)
Calculate parking fee
and apply daily maximum limit
Is entry date ≠ exit date
(overnight parking)?
Yes
Add overnight charge
(₹50)
No
Is duration > 24 hours?
Yes
Add penalty charge
(₹100)
No
Calculate total amount
Display receipt
(duration, charges, total)
Stop

6. Pseudocode:

```
START

DISPLAY "SMART PARKING FEE MANAGEMENT"

INPUT vehicle_type
INPUT entry_date_time
INPUT exit_date_time

IF vehicle_type is invalid
   DISPLAY "Invalid vehicle type"
   STOP
END IF

VALIDATE entry_date_time and exit_date_time

IF exit_date_time <= entry_date_time
   DISPLAY "Invalid exit time"
   STOP
END IF

CALCULATE parking_duration

CALCULATE total_hours by rounding up

GET hourly_rate and daily_limit based on vehicle_type

CALCULATE daily_blocks = ceiling(total_hours / 24)

CALCULATE parking_fee = total_hours * hourly_rate

CALCULATE maximum_fee = daily_blocks * daily_limit

IF parking_fee > maximum_fee
   parking_fee = maximum_fee
END IF

IF entry date is different from exit date
   ADD overnight charge
END IF

IF parking_duration > 24 hours
   ADD penalty charge
END IF

CALCULATE final_amount
```

```
DISPLAY vehicle_type
DISPLAY parking_duration
DISPLAY parking_fee
DISPLAY overnight_charge
DISPLAY penalty_charge
DISPLAY final_amount

STOP
```

7. Python Source Code:

```
from datetime import datetime
import math
rates = {
    "bike": [10, 100],
    "car": [20, 200],
    "bus": [30, 300]
}
vehicle = input("Enter vehicle type (Bike/Car/Bus): ").lower()

if vehicle not in rates:
    print("Invalid vehicle type!")

else:
    try:
        entry = input("Enter entry (YYYY-MM-DD HH:MM): ")
        exit = input("Enter exit (YYYY-MM-DD HH:MM): ")

        entry_time = datetime.strptime(entry, "%Y-%m-%d %H:%M")
        exit_time = datetime.strptime(exit, "%Y-%m-%d %H:%M")

        if exit_time <= entry_time:
            print("Invalid time! Exit must be after entry.")

        Else:
            duration = exit_time - entry_time
            hours = math.ceil(duration.total_seconds() / 3600
            rate = rates[vehicle][0]
            daily_limit = rates[vehicle][1]
            days = math.ceil(hours / 24)
            parking_fee = hours * rate
            maximum_fee = days * daily_limit

            if parking_fee > maximum_fee:
                parking_fee = maximum_fee
```

```
        overnight_fee = 0

        if entry_time.date() != exit_time.date():
            overnight_fee = 50
        penalty = 0

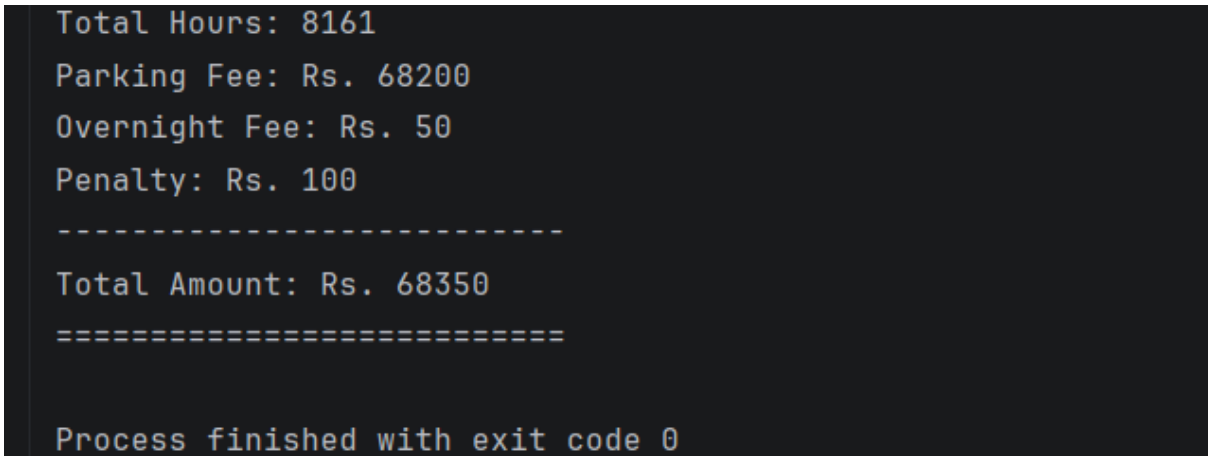
        if hours > 24:
            penalty = 100
        total = parking_fee + overnight_fee + penalty
        print("Vehicle Type:", vehicle.capitalize())
        print("Parking Duration:", duration)
        print("Total Hours:", hours)
        print("Parking Fee: Rs.", parking_fee)
        print("Overnight Fee: Rs.", overnight_fee)
        print("Penalty: Rs.", penalty)
        print("Total Amount: Rs.", total
    except ValueError:
        print("Invalid date or time format!")
```

8. Test Cases:

#1
Enter vehicle type (Bike/Car/Bus): bike
Enter entry (YYYY-MM-DD HH:MM): 2026-10-30 10:00
Enter exit (YYYY-MM-DD HH:MM): 2026-10-30 12:00
Vehicle Type: Bike
Parking Duration: 2:00:00
Total Hours: 2
Parking Fee: Rs. 20
Overnight Fee: Rs. 0
Penalty: Rs. 0
Total Amount: Rs. 20
#2
Vehicle Type: Car
Parking Duration: 4:00:00
Total Hours: 4
Parking Fee: Rs. 80
Overnight Fee: Rs. 0
Penalty: Rs. 0
Total Amount: Rs. 80
#3
Vehicle Type: Bike
Parking Duration: 12:00:00
Total Hours: 12
Parking Fee: Rs. 100

Overnight Fee: Rs. 0
Penalty: Rs. 0
Total Amount: Rs. 100

9.output screenshot:



10.conclusion:

The Smart Parking Fee Management System calculates parking charges accurately based on vehicle type and parking duration. It handles overnight charges, daily limits, and penalties while validating time inputs. This project makes parking fee calculation simple, fast, and efficient using basic Python programming.

Case study 8:

Student Timetable Conflict Detection: Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

1. Problem Statement

The Student Timetable Conflict Detection System is designed to identify scheduling conflicts in a student's proposed timetable. The system accepts course names, days, starting times, ending times, and room numbers.

It compares course schedules represented using strings and tuples to detect overlapping class periods. It also checks whether the selected classrooms are unavailable during the proposed class timings.

The system validates the proposed timetable and displays whether it is valid or contains conflicts. The main aim is to help students prepare a conflict-free timetable and avoid overlapping classes and room allocation problems.

2. Objectives

1. To accept course schedules using strings and tuples.
2. To identify overlapping class timings.
3. To detect conflicts between different courses.
4. To identify unavailable classrooms.
5. To validate starting and ending times.
6. To compare schedules based on days and time periods.
7. To display detected timetable conflicts.
8. To generate a valid timetable without scheduling errors.

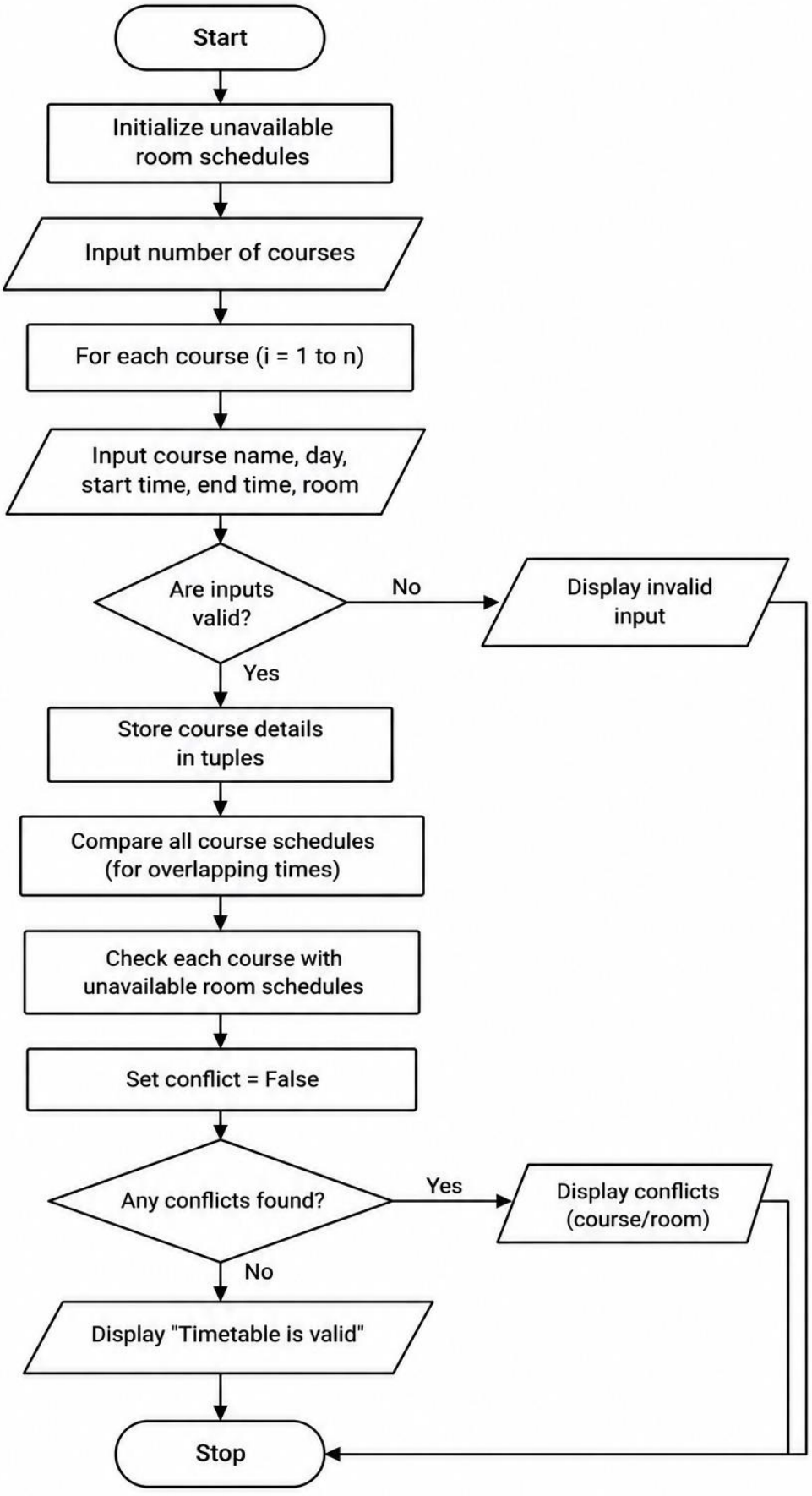
3. Assumptions

1. Each course has a course name, day, start time, end time, and room number.
2. Days are represented as Monday to Saturday.
3. Time is entered in 24-hour format (HH:MM).
4. Two classes conflict if their timings overlap on the same day.
5. Classes ending exactly when another class starts are not considered conflicts.
6. A room cannot be used when it is marked unavailable.
7. Class duration must be greater than zero.
8. The system checks the proposed timetable against predefined unavailable room schedules.

4. Algorithm

1. Start the program.
2. Initialize the unavailable room schedules.
3. Accept the number of courses.
4. Read the course name, day, start time, end time, and room.
5. Validate the day and time inputs.
6. Check whether the start time is earlier than the end time.
7. Store the course details in tuples.
8. Compare every pair of courses.
9. Check whether two courses occur on the same day.
10. Check whether their time periods overlap.
11. Display a course conflict if overlapping periods are found.
12. Compare each proposed course with unavailable room schedules.
13. Display a room conflict if the selected room is unavailable.
14. If no conflicts are found, display "Timetable is valid".
15. Otherwise, display the detected conflicts.
16. Stop the program

5. Flowchart:


Start
Initialize unavailable room schedules
Input number of courses
For each course (i = 1 to n)
Input course name, day, start time, end time, room
Are inputs valid?
No
Display invalid input
Yes
Store course details in tuples
Compare all course schedules (for overlapping times)
Check each course with unavailable room schedules
Set conflict = False
Any conflicts found?
Yes
Display conflicts (course/room)
No
Display "Timetable is valid"
Stop

6. Pseudocode:

```
START

INITIALIZE unavailable_rooms

INPUT number_of_courses

FOR each course:
   INPUT course_name
   INPUT day
   INPUT start_time
   INPUT end_time
   INPUT room_number

   VALIDATE inputs

   STORE course as tuple
END FOR

SET conflict = FALSE

FOR each pair of courses:
   IF both courses have the same day:
      IF their timings overlap:
         DISPLAY course conflict
         SET conflict = TRUE
      END IF
   END IF
END FOR

FOR each course:
   CHECK unavailable room schedules

   IF room is unavailable:
      DISPLAY room conflict
      SET conflict = TRUE
   END IF
END FOR

IF conflict == FALSE:
   DISPLAY "Timetable is valid"
ELSE:
   DISPLAY "Timetable contains conflicts"
END IF

STOP
```

7. Python Source Code:

```
unavailable_rooms = [
    ("R103", "Tuesday", "10:00", "11:00"),
    ("R105", "Monday", "09:00", "12:00")
]
courses = []

days = ["Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday"]

n = int(input("Enter number of courses: "))
for i in range(n):
    print("\nEnter details for course", i + 1)

    name = input("Course name: ")
    day = input("Day: ").capitalize()
    start = input("Start time (HH:MM): ")
    end = input("End time (HH:MM): ")
    room = input("Room number: ")

    try:
        start_minutes = int(start[:2]) * 60 + int(start[3:])
        end_minutes = int(end[:2]) * 60 + int(end[3:])

        valid_start = (
            len(start) == 5 and start[2] == ":"
            and 0 <= int(start[:2]) < 24
            and 0 <= int(start[3:]) < 60
        )

        valid_end = (
            len(end) == 5 and end[2] == ":"
            and 0 <= int(end[:2]) < 24
            and 0 <= int(end[3:]) < 60
        )

        if (day not in days or not valid_start
                or not valid_end or start_minutes >= end_minutes):
            print("Invalid schedule details!")
            continue

        courses.append((name, day, start, end, room))

    except ValueError:
        print("Invalid time format!")
conflict = False
```

```
for i in range(len(courses)):
    for j in range(i + 1, len(courses)):

        c1 = courses[i]
        c2 = courses[j]

        s1 = int(c1[2][:2]) * 60 + int(c1[2][3:])
        e1 = int(c1[3][:2]) * 60 + int(c1[3][3:])

        s2 = int(c2[2][:2]) * 60 + int(c2[2][3:])
        e2 = int(c2[3][:2]) * 60 + int(c2[3][3:])

        if c1[1] == c2[1] and s1 < e2 and s2 < e1:
            print("Course conflict:", c1[0], "and", c2[0])
            conflict = True
for course in courses:
    for room in unavailable_rooms:

        s1 = int(course[2][:2]) * 60 + int(course[2][3:])
        e1 = int(course[3][:2]) * 60 + int(course[3][3:])

        s2 = int(room[2][:2]) * 60 + int(room[2][3:])
        e2 = int(room[3][:2]) * 60 + int(room[3][3:])

        if (course[4] == room[0] and course[1] == room[1]
                and s1 < e2 and s2 < e1):
            print("Room conflict:", course[0],
                  "Room", course[4])
            conflict = True
if conflict:
    print("\nTimetable contains conflicts!")
else:
    print("\nTimetable is valid!")
```

8. Test Cases:

#1

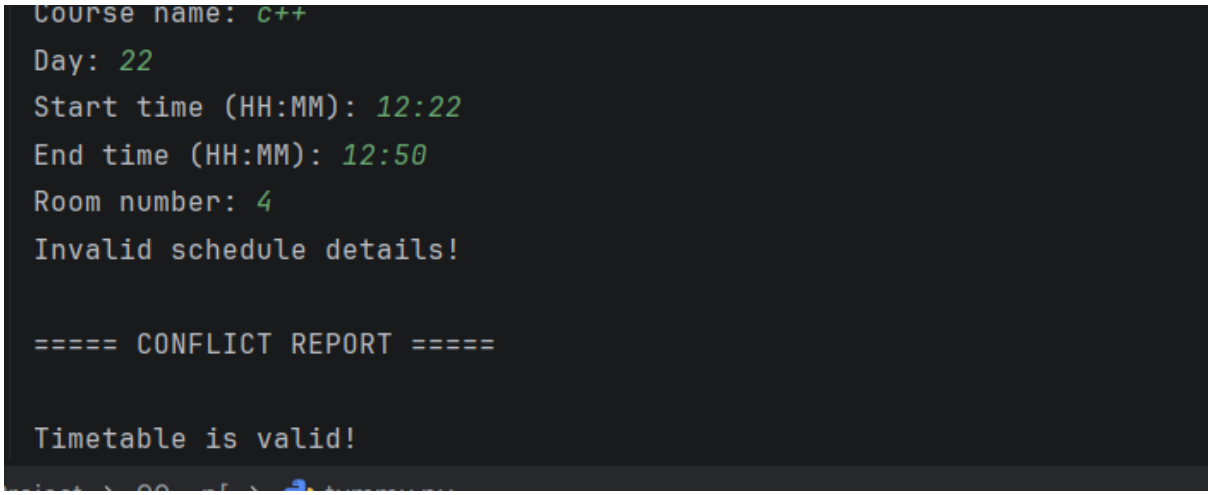
```
Enter number of courses: 2

Enter details for course 1
Course name: Python
Day: Monday
Start time (HH:MM): 09:00
End time (HH:MM): 10:00
Room number: R101

Enter details for course 2
```

Course name: Java
Day: Monday
Start time (HH:MM): 09:30
End time (HH:MM): 10:30
Room number: R102
Course conflict: Python and Java
Timetable contains conflicts!
#2
Enter number of courses: 1
Enter details for course 1
Course name: DBMS
Day: Tuesday
Start time (HH:MM): 10:00
End time (HH:MM): 11:00
Room number: R103
Room conflict: DBMS Room R103
Timetable contains conflicts!
#3
Enter number of courses: 2
Enter details for course 1
Course name: Python
Day: Monday
Start time (HH:MM): 09:00
End time (HH:MM): 10:00
Room number: R101
Enter details for course 2
Course name: Java
Day: Monday
Start time (HH:MM): 10:00
End time (HH:MM): 11:00
Room number: R102
Timetable is valid!

9.output screenshot:



10.conclusion:

The Student Timetable Conflict Detection System identifies overlapping class timings and unavailable classrooms. It validates course schedules and helps students create conflict-free timetables. This project demonstrates the use of Python strings, tuples, lists, loops, and conditional statements to solve scheduling problems easily and efficiently.

Case study 9:

Progressive Water Tariff Analysis: Calculate bills for multiple households using consumption slabs, fixed fees, and penalties; compare consumption patterns and identify excessive usage.

1. Problem Statement

The Progressive Water Tariff Analysis System is designed to calculate water bills for multiple households based on their monthly water consumption. The system uses different tariff rates for different consumption slabs.

It calculates the water charges according to the units consumed, adds fixed service fees, and applies penalties when consumption exceeds the specified limit. It also compares water consumption among households and identifies households with excessive water usage.

The main aim is to calculate accurate water bills and encourage households to reduce unnecessary water consumption.

2. Objectives

1. To calculate water bills for multiple households.
2. To apply different rates for different consumption slabs.
3. To add fixed charges to the water bill.
4. To apply penalties for excessive water consumption.
5. To compare water usage among households.
6. To identify households with high water consumption.
7. To display individual household bills.
8. To calculate the total water bill for all households.

3. Assumptions

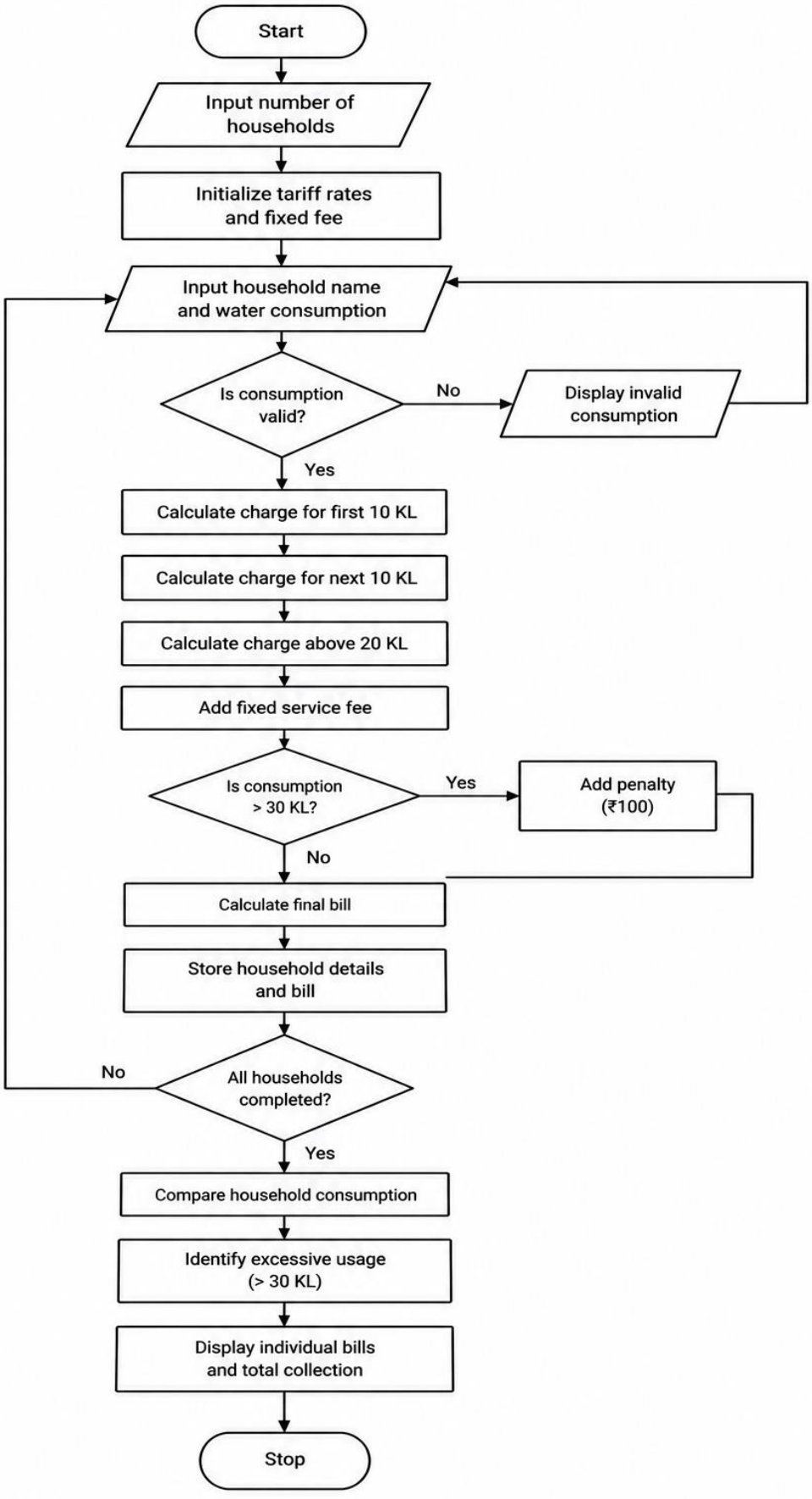
1. Water consumption is measured in kilolitres (KL).
2. The system accepts multiple household details.
3. The first 10 KL costs ₹5 per KL.
4. The next 10 KL costs ₹8 per KL.

5. Consumption above 20 KL costs ₹12 per KL.
6. A fixed service charge of ₹50 is added to every bill.
7. A penalty of ₹100 is charged if consumption exceeds 30 KL.
8. Consumption above 30 KL is considered excessive usage.
9. Negative consumption values are not accepted.
10. All calculations are performed monthly.

4. Algorithm

1. Start the program.
2. Accept the number of households.
3. Initialize the tariff rates and fixed fee.
4. Read the household name and water consumption.
5. Validate the consumption value.
6. Calculate the charge for the first 10 KL.
7. Calculate the charge for the next 10 KL.
8. Calculate the charge above 20 KL.
9. Add the fixed service fee.
10. Check whether consumption exceeds 30 KL.
11. If consumption exceeds 30 KL, add a ₹100 penalty.
12. Calculate the final water bill.
13. Store household names, consumption, and bills.
14. Compare the consumption of all households.
15. Identify households with excessive water usage.
16. Display individual bills and consumption details.
17. Display total water collection and average consumption.
18. Stop the program.

5. Flowchart:


Start
Input number of households
Initialize tariff rates and fixed fee
Input household name and water consumption
Is consumption valid?
No
Display invalid consumption
Yes
Calculate charge for first 10 KL
Calculate charge for next 10 KL
Calculate charge above 20 KL
Add fixed service fee
Is consumption > 30 KL?
Yes
Add penalty (₹100)
No
Calculate final bill
Store household details and bill
All households completed?
No
Yes
Compare household consumption
Identify excessive usage (> 30 KL)
Display individual bills and total collection
Stop

6. Pseudocode:

```
START

INPUT number_of_households

INITIALIZE total_bill = 0

FOR each household:

    INPUT household_name
    INPUT water_consumption

    IF water_consumption < 0:
        DISPLAY "Invalid consumption"
        CONTINUE
    END IF

    IF consumption <= 10:
        charge = consumption * 5

    ELSE IF consumption <= 20:
        charge = 10 * 5 + (consumption - 10) * 8

    ELSE:
        charge = 10 * 5 + 10 * 8
                + (consumption - 20) * 12
    END IF

    ADD fixed fee of 50

    IF consumption > 30:
        ADD penalty of 100
        DISPLAY "Excessive usage"
    END IF

    CALCULATE final bill

    STORE household details

    ADD bill to total_bill

END FOR

COMPARE household consumption

DISPLAY individual bills
DISPLAY excessive usage households
```

DISPLAY total bill collection

STOP

7. Python Source Code:

```
households = []
total_bill = 0

n = int(input("Enter number of households: "))

for i in range(n):

    print("\nHousehold", i + 1)

    name = input("Enter household name: ")
    units = float(input("Enter water consumption (KL): "))

    if units < 0:
        print("Invalid consumption!")
        continue
    if units <= 10:
        charge = units * 5

    elif units <= 20:
        charge = (10 * 5) + (units - 10) * 8

    else:
        charge = (10 * 5) + (10 * 8) + (units - 20) * 12
    fixed_fee = 50
    penalty = 0

    if units > 30:
        penalty = 100
    bill = charge + fixed_fee + penalty

    households.append((name, units, bill))

    total_bill = total_bill + bill

for h in households:

    print("\nHousehold:", h[0])
    print("Water Consumption:", h[1], "KL")
```

```
    print("Water Charge: Rs.", h[2] - 50 -
          (100 if h[1] > 30 else 0))
    print("Fixed Fee: Rs. 50")

    if h[1] > 30:
        print("Penalty: Rs. 100")
        print("Excessive Usage!")

    else:
        print("Penalty: Rs. 0")

    print("Total Bill: Rs.", h[2])

for h in households:

    if h[1] > 30:
        print(h[0], "- Excessive Water Usage")
    else:
        print(h[0], "- Normal Water Usage")
if len(households) > 0:

    total_units = 0

    for h in households:
        total_units = total_units + h[1]

    average = total_units / len(households)

    print("\nAverage Consumption:", average, "KL")

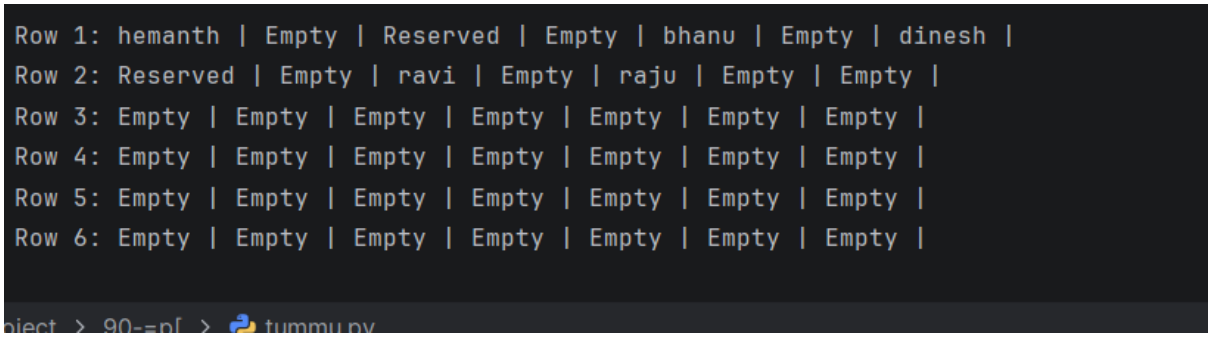
print("Total Water Bill Collection: Rs.", total_bill)
```

8. Test Cases:

#1
Enter number of households: 1
Household 1
Enter household name: House A
Enter water consumption (KL): 5
Household: House A
Water Consumption: 5.0 KL
Water Charge: Rs. 25.0
Fixed Fee: Rs. 50
Penalty: Rs. 0
Total Bill: Rs. 75.0
House A - Normal Water Usage

Average Consumption: 5.0 KL
Total Water Bill Collection: Rs. 75.0
#2
Household: House B
Water Consumption: 10.0 KL
Water Charge: Rs. 50.0
Fixed Fee: Rs. 50
Penalty: Rs. 0
Total Bill: Rs. 100.0
#3
Household: House C
Water Consumption: 15.0 KL
Water Charge: Rs. 90.0
Fixed Fee: Rs. 50
Penalty: Rs. 0
Total Bill: Rs. 140.0
#4
Enter household name: House H
Enter water consumption (KL): -5
Invalid consumption!

9.output screenshot:



10.conclusion:

The Progressive Water Tariff Analysis System calculates water bills accurately using consumption slabs, fixed fees, and penalties. It compares household water usage and identifies excessive consumption. This project demonstrates how basic Python concepts can be used to manage water billing easily and efficiently.

Case study 10:

Nested Loop Examination Seating Generator: Generate a seating arrangement with row and column identifiers, reserved positions, alternate seating restrictions, and fixed classroom capacities.

1. Problem Statement

The Nested Loop Examination Seating Generator is designed to generate seating arrangements for students in an examination hall. The system accepts the number of rows, columns, and student names to create a structured seating arrangement.

It uses nested loops to assign seats with row and column identifiers. It also considers reserved seating positions, alternate seating restrictions, and the fixed classroom capacity.

The system ensures that students are assigned seats without using reserved positions and maintains alternate seating gaps to reduce the chances of copying during examinations.

2. Objectives

1. To generate examination seating arrangements automatically.
2. To assign unique row and column identifiers.
3. To identify and maintain reserved seating positions.
4. To implement alternate seating restrictions.
5. To calculate the total classroom capacity.
6. To prevent seating beyond the available capacity.
7. To display the seating arrangement clearly.
8. To use nested loops for seat allocation.

3. Assumptions

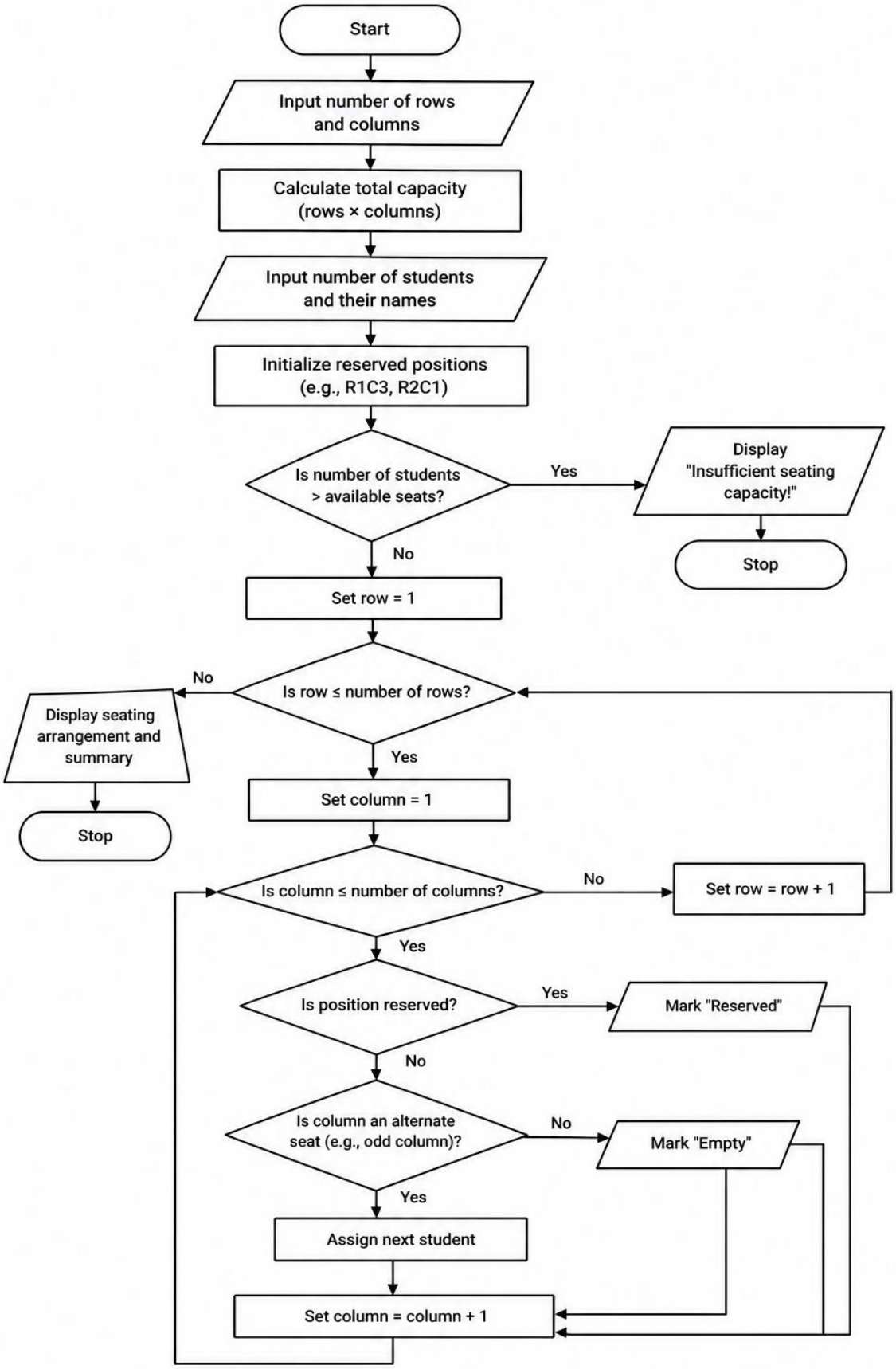
1. The examination hall contains a fixed number of rows and columns.
2. Each position can accommodate only one student.
3. Alternate seating is followed by leaving one seat empty between occupied seats.
4. Some positions are reserved and cannot be assigned to students.
5. Reserved positions are entered using row and column numbers.
6. Students are assigned seats from left to right and top to bottom.
7. The total classroom capacity is calculated as rows multiplied by columns.
8. If the number of students exceeds the available seats, the system displays an error message.
9. Row and column numbering starts from 1.

4. Algorithm

1. Start the program.
2. Input the number of rows and columns.
3. Calculate the total classroom capacity.
4. Input the number of students.
5. Accept student names.
6. Initialize reserved seating positions.
7. Check whether the number of students exceeds the available seating capacity.
8. Use an outer loop to represent rows.

9. Use an inner loop to represent columns.
10. Check whether the current position is reserved.
11. Check whether the column follows alternate seating restrictions.
12. If the position is available, assign a student.
13. Otherwise, mark the position as Reserved or Empty.
14. Continue until all rows and columns are checked.
15. Display the complete seating arrangement.
16. Display the total capacity and allocated seats.
17. Stop the program.

5. Flowchart:


Start
Input number of rows and columns
Calculate total capacity (rows × columns)
Input number of students and their names
Initialize reserved positions (e.g., R1C3, R2C1)
Is number of students > available seats?
Yes
Display "Insufficient seating capacity!"
Stop
No
Set row = 1
Is row ≤ number of rows?
No
Display seating arrangement and summary
Stop
Yes
Set column = 1
Is column ≤ number of columns?
No
Set row = row + 1
Yes
Is position reserved?
Yes
Mark "Reserved"
No
Is column an alternate seat (e.g., odd column)?
No
Mark "Empty"
Yes
Assign next student
Set column = column + 1

6. Pseudocode:

```
START

INPUT number_of_rows
INPUT number_of_columns

CALCULATE total_capacity

INPUT number_of_students
INPUT student_names

INITIALIZE reserved_positions

CALCULATE available_seats

IF students > available_seats:
   DISPLAY "Insufficient seats"
   STOP
END IF

FOR each row:
   FOR each column:

      IF position is reserved:
         DISPLAY "Reserved"

      ELSE IF column is not an alternate seat:
         DISPLAY "Empty"

      ELSE IF students are available:
         ASSIGN next student
      ELSE:
         DISPLAY "Empty"

      END IF

   END FOR
END FOR

DISPLAY seating arrangement
DISPLAY total capacity
DISPLAY allocated seats

STOP
```

7. Python Source Code:

```
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

capacity = rows * cols

print("Total Classroom Capacity:", capacity)
n = int(input("Enter number of students: "))

students = []

for i in range(n):
    name = input("Enter student name: ")
    students.append(name)
reserved = {(1, 3), (2, 1)}
available = 0

for i in range(1, rows + 1):
    for j in range(1, cols + 1):

        if j % 2 != 0 and (i, j) not in reserved:
            available += 1
if n > available or n < 0:
    print("Insufficient seating capacity!")

else:
    count = 0
    allocated = 0
    for i in range(1, rows + 1):

        print("\nRow", i, end=": ")

        for j in range(1, cols + 1):

            if (i, j) in reserved:
                print("Reserved", end=" | ")

            elif j % 2 == 0:
                print("Empty", end=" | ")

            elif count < n:
                print(students[count], end=" | ")
                count += 1
                allocated += 1

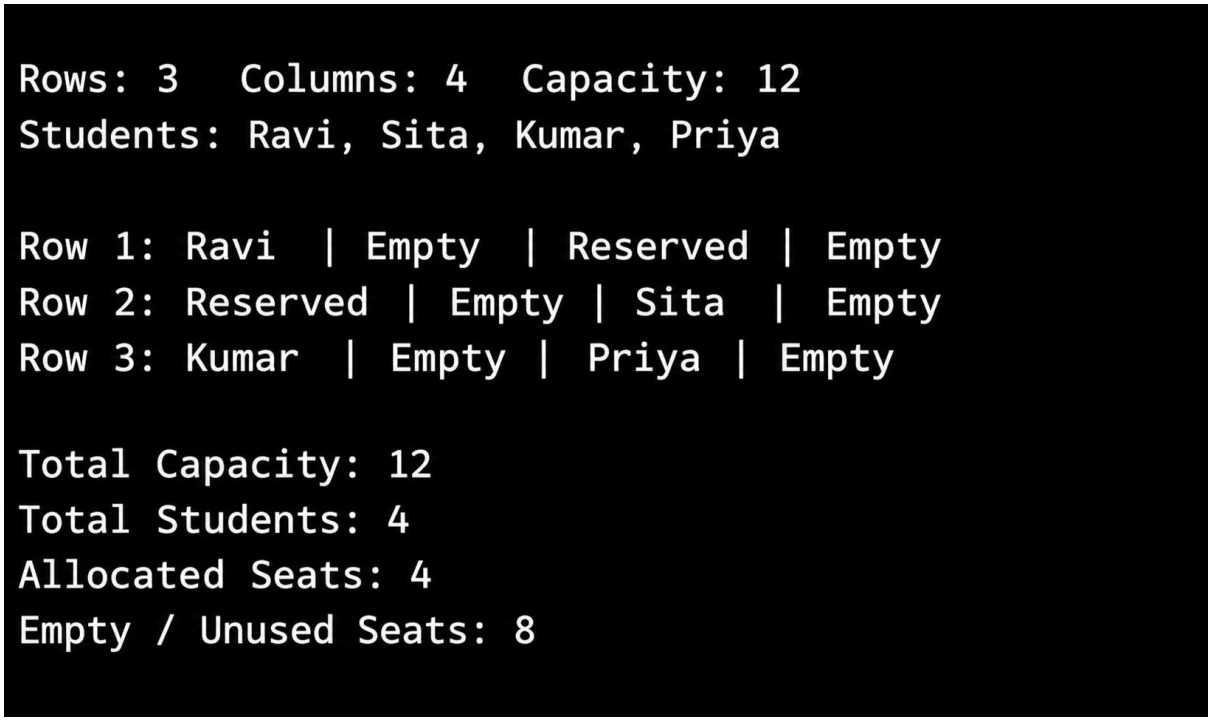
            else:
                print("Empty", end=" | ")
```

```
    print("Total Capacity:", capacity)
    print("Total Students:", n)
    print("Allocated Seats:", allocated)
    print("Empty / Unused Seats:", capacity - allocated)
    print("Reserved Positions:", reserved)
```

8. Test Cases:

#1
Enter number of rows: 3
Enter number of columns: 4
Total Classroom Capacity: 12
Enter number of students: 4
Enter student name: Ravi
Enter student name: Sita
Enter student name: Kumar
Enter student name: Priya
Row 1: Ravi | Empty | Reserved | Empty |
Row 2: Reserved | Empty | Sita | Empty |
Row 3: Kumar | Empty | Priya | Empty |
Total Capacity: 12
Total Students: 4
Allocated Seats: 4
Empty / Unused Seats: 8
Reserved Positions: {(1, 3), (2, 1)}
#2
Enter number of rows: 3
Enter number of columns: 4
Total Classroom Capacity: 12
Enter number of students: 5
Row 1: Student1 | Empty | Reserved | Empty |
Row 2: Reserved | Empty | Student2 | Empty |
Row 3: Student3 | Empty | Student4 | Empty |
Total Capacity: 12
Total Students: 5
Allocated Seats: 5
#3
Row 1: Ravi | Empty | Reserved | Empty |
Row 2: Reserved | Empty | Sita | Empty |
Row 3: Kumar | Empty | Priya | Empty |

9.output screenshot:



10.conclusion:

The Nested Loop Examination Seating Generator automatically creates examination seating arrangements using rows and columns. It handles reserved positions, alternate seating restrictions, and classroom capacity. This project demonstrates the use of Python nested loops, lists, tuples, and conditional statements to generate organized and conflict-free seating arrangements.